Using R, Seurat, and CellChat to Analyze a Single-Cell Transcriptomics Dataset of Mouse Skin Wound Healing

Shalyn Keiser[1], **Nichole Botello**[1], **Ethan Cruz**[1], **Mateusz S. Wietecha**[1]

[1] Department of Oral Biology, College of Dentistry, University of Illinois Chicago

Corresponding Author

Mateusz S. Wietecha
mwietec2@uic.edu

Citation

Keiser, S., Botello, N., Cruz, E., Wietecha, M.S. Using R, Seurat, and CellChat to Analyze a Single-Cell Transcriptomics Dataset of Mouse Skin Wound Healing. *J. Vis. Exp.* (222), e67266, doi:10.3791/67266 (2025).

Date Published

August 1, 2025

DOI

10.3791/67266

URL

jove.com/video/67266

Abstract

The process of wound healing is regulated by complex interactions between different cell types across space and time. Through the profiling of individual cells within their complex environment, single-cell transcriptomics methods enable the investigation of cellular heterogeneity, cell communication networks, and cell-cell interactions involved in the wound healing process. However, many single-cell analysis tools are run within a computer coding environment, and their more widespread use by wound healing scientists is thwarted by the apparent lack of bioinformatics expertise. Therefore, a step-by-step workflow is presented showing how to use a graphical coding environment called RStudio to perform a basic single-cell analysis of a temporal mouse excisional skin wound healing dataset. This visual and guided protocol will enable scientists with no bioinformatics background to download a previously published wound healing dataset, perform critical quality control steps, run a standard single-cell analysis workflow including dataset visualizations and cell type annotations using Seurat, run cell subtype analyses, run module scoring analyses, run cell-cell interaction analyses using CellChat, and perform integrative analyses of multiple datasets using Seurat. Narrative explanations are provided for each step in the protocol and graphical results from every line of code are presented to safely guide the user through the workflow. The goal of this visual introduction to a single-cell analysis pipeline is to enable more wound healing scientists to use bioinformatics tools directly in their own laboratories in order to facilitate deeper analyses of their own single-cell datasets as well as more widespread re-analyses of previously published single-cell datasets.

Introduction

Wound healing is one of the most complex processes in mammalian biology, and it involves a spectrum of three phases of healing: Inflammatory, Proliferative, and Resolution[1,2]. These phases of healing broadly classify the

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

coordinated actions of dozens of cell types and hundreds of their molecular products across space and time of wound repair[3]. Several decades of histological and molecular studies based on wound tissue sampling across the time-course of healing have elucidated the overarching cellular patterns of tissue repair[3], particularly in reproducible mouse models of excisional skin wound healing[4,5,6]. Only in the last two decades has it become possible to more fully appreciate the complexity of wound healing, beginning with the advent of high-throughput transcriptomic analyses of wounds at the scales of bulk tissues[7,8,9] and cells[10,11,12,13,14]. Most recently, several studies have transcriptionally profiled skin wounds at the single-cell level, identifying new wound cell subtypes and showing how they may interact with one another during healing[15,16,17,18,19,20]. Hu et al. used an innovative spatial single-cell RNA sequencing approach to profile skin wounds throughout the time-course of healing at several radial distances from the wound center, which revealed new inter-cellular and molecular 'movements' across space and time[20]. Such studies are unraveling the complexity of wound healing in unprecedented detail, and they are beginning to paint a picture of tremendous cellular and molecular heterogeneity.

Major recent advances in bioinformatics analysis methods are making it possible to make biological sense of the complex multi-omic datasets being generated in the field of wound healing research. Single-cell analysis packages like Seurat provide tools for robust analysis and integration of datasets, including the classification of cell types in complex tissues such as wounds[21]. For downstream interpretation of single-cell data, tools like CellChat are used to identify putative cell-cell interaction programs that may explain how cells coordinate to repair wounds[22]. While these tools are well-documented and well-cited, they must be run within a computer coding environment such as R, a statistical and graphical programming language that is most commonly used in the bioinformatics fields of genomics and transcriptomics. While biologists and clinicians in the wound healing field are increasingly using single-cell approaches to study tissue repair, few have the bioinformatics training required to use tools such as Seurat and CellChat directly in their own labs. Such a barrier in the use of these bioinformatics tools not only prevents scientists from more deeply analyzing their own datasets without the help of bioinformaticians, but it also prevents scientists from reliably re-analyzing the wealth of single-cell data already published by other groups.

Therefore, a step-by-step workflow is presented here to enable scientists with no bioinformatics background to analyze a previously published and publicly available single-cell wound healing dataset[20]. The protocol uses the prevalent and free graphical R coding environment called RStudio, and it demonstrates how to navigate this environment to run prescribed lines of code that enable a basic analysis of a complex single-cell dataset using Seurat and CellChat. Within this protocol, the seven major methods relevant for wound healing research are presented, including: 1) coding environment installation, 2) downloading the dataset and critical quality control steps, 3) single-cell analysis workflows including visualizations and cell type annotations, 4) cell subtype analyses, 5) module scoring analyses, 6) cell-cell interaction analyses, and 7) integrative analyses of multiple datasets. Within each method, actual code is provided for the user to run side-by-side with the protocol, and actual graphical results from every line of code are shown to guide the user through the workflow. The primary goal of this guided and visual introduction to RStudio and a fundamental single-cell analysis workflow is to enable more wound healing scientists

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

to use these powerful tools directly in order to enable faster progress in the field of research.

Protocol

NOTE: In the following workflows detailing seven bioinformatics methods, all steps of the protocols are accompanied by their respective blocks of code that should be run directly on the user's own RStudio interface in the order that they are listed. To make this protocol as user-friendly as possible, an R script file is included (**Supplementary File 1: JoVE_Rscript.R**), which may be loaded directly into the user's RStudio session, so that each line of code may simply be run. This avoids the user having to type or copy and paste the code from the protocol document, which could introduce errors. All protocol instructions are also included in the R script file in the form of comments, indicated by a hashtag '#' symbol at the beginning of each comment line.

1. Installing R, RStudio, and the required R packages for the single-cell analysis workflow

1. Download and install R (version 4.4.1) to the computer. Use the link that corresponds to the computer's operating system.
    1. If using a computer running Microsoft Windows, use this link: https://cran.rstudio.com/bin/windows/base/
    2. If using a computer running MacOS, use this link: https://cran.rstudio.com/bin/macosx/
2. Install the latest version of RStudio on the computer. Click on the following link and follow the instructions: https://www.rstudio.com/products/rstudio/download/#download
3. Install the Rtools (version 4.4), which will allow R to compile certain packages. Click on the following link and follow the instructions:
    1. If using Windows, use this link: https://cran.rstudio.com/bin/windows/Rtools/
    2. If using MacOS, use this link: https://mac.r-project.org/tools/
4. Set the local working directory; this is the folder on the computer where all files will be loaded in from and saved into. Set the working directory by selecting **Session** in the RStudio menu bar and clicking **Set Working Directory > Choose Directory** and selecting the desired folder.
    1. If using a Windows computer, use the following command to set the working directory. Change [Directory] in the following line of code to the actual directory structure. Be mindful that the directory delimiter in R is the character "/"

    ```
    setwd("C:/[Directory]")
    ```

    2. If using a MacOS computer, the following command will also set the working directory. Change [Directory] in the following line of code to the actual directory structure. Be mindful that the directory delimiter in R is the character "/"

    ```
    setwd("~/[Directory]")
    ```

    3. At any point during an R session, check the working directory using the following line of code:

    ```
    getwd()
    ```

    4. In RStudio, visually explore the working directory structure, including all files and folders contained therein, within the right-hand window in the **Files** tab.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

To navigate RStudio's Files explorer to the working directory, click the **gear** icon -> **Go To Working Directory**.

5. Install the following packages from the R package repository CRAN, which are necessary dependencies for the protocol. To install these packages, run the following commands.

```
install.packages("devtools")
 install.packages("readxl")
 install.packages("openxlsx")
 install.packages("tidyverse")
 install.packages("scCustomize")
```

NOTE: During installation of R packages, it is normal for various windows to appear and disappear. If a window appears asking to compile a package, click **YES**. If a window appears asking to restart R prior to installing the package, click **NO**.

6. Install the following packages from the curated R package repository Bioconductor (https://bioconductor.org/), which are necessary dependencies for the protocol. To install these packages, run the following commands.

```
if (! requireNamespace("BiocManager",
 quietly = TRUE) )
   install.packages("BiocManager")
 BiocManager::install("NMF", update=F)
 BiocManager::install("ComplexHeatmap",
 update=F)
 BiocManager::install("BiocNeighbors",
 update=F)
 BiocManager::install("SingleCellExperiment",
 update=F)
 BiocManager::install("circlize",
 update=F)
 BiocManager::install("edgeR", update=F)
 BiocManager::install("scDblFinder",
 update=F)
```

7. Install the following packages, which are required for the workflow described in this manuscript.

```
install.packages("Seurat")
 devtools::install_github("jinworks/
CellChat")
```

8. Load in each package to confirm that the installations were successful. In case any of the packages results in a "package not found" error, re-install it using the appropriate code above.

```
library(readxl)
 library(openxlsx)
 library(tidyverse)
 library(scCustomize)
 library(edgeR)
 library(scDblFinder)
 library(Seurat)
 library(CellChat)
```

2. Loading in a single-cell wound healing dataset and performing quality control steps

NOTE: For this bioinformatics workflow, a re-analysis of a previously published spatio-temporal single-cell skin wound healing experiment is conducted[20]. The dataset files are stored in the curated NCBI Gene Expression Omnibus (GEO) repository (https://www.ncbi.nlm.nih.gov/geo/).

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

1. Navigate to the dataset files from GEO using the accession number GSE204777. Use the following link and review the study's experimental design: https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE204777

2. Within the GEO repository page, there are five individual batches of data obtained from five sequencing lanes. Click on the first dataset, entitled GSM6190913. The following is a direct link to the sample: https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSM6190913

3. Scroll down to the bottom of the page and download the following three files, using either the **ftp** or **html** links. Within the computer's file explorer, move these three files into a directory called **b1**. Make sure that the **b1** folder is located within the working directory set in Step 1.4.
File name: GSM6190913_b1_barcodes.tsv.gz / file size: 18.5 Mb
File name: GSM6190913_b1_features.tsv.gz / file size: 254.1 Kb
File name: GSM6190913_b1_matrix.mtx.gz / file size: 151.2 Mb

4. Get the directory information for the single-cell sequencing files downloaded in step 2.3.

```
b1_data_dir <- file.path(getwd(), "b1")
```

5. Load in the single-cell sequencing files. The gene.column parameter specifies the gene/feature naming used. In this case, use gene.column = 2 for gene symbols (gene.column = 1 is for Ensembl gene names).

```
b1 <- Read10X_GEO(data_dir = b1_data_dir,
 gene.column = 2)
```

NOTE: Most single-cell datasets do not have additional multiplexed data, so the 10x files generated using this step would not have multiple layers. For the current multiplexed dataset, proceed with step 2.6. For datasets without multiplexing data, skip to step 2.7.

6. Demultiplex the single-cell dataset using spatio-temporal barcodes.

1. For the working dataset, separate the gene expression and HTO (multiplexing) data.

```
dataset_rna_counts <-
 b1$GSM6190913_b1_$`Gene Expression`
 dataset_barcodes <-
 b1$GSM6190913_b1_$`Antibody Capture`
```

2. Create a Seurat object using the gene expression data, while immediately filtering out genes expressed in less than 5 cells and cells with less than 200 genes detected.

```
dataset <- CreateSeuratObject(counts
 = dataset_rna_counts, min.cells = 5,
 min.features = 200)
```

3. Create a gene expression dataset as a layer and generate a list of cells and barcodes common to both assays.

```
dataset_rna_counts2 <-
 LayerData(dataset, data = "RNA")
 dataset_joint.barcodes <-
 intersect(colnames(dataset_rna_counts2),
 colnames(dataset_barcodes))
```

4. Subset gene expression and HTO counts by joint cell barcodes.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
dataset_rna_counts2 <-
 dataset_rna_counts2[,
 dataset_joint.barcodes]
 dataset_barcodes2 <-
 as.matrix(dataset_barcodes[,
 dataset_joint.barcodes])
```

5. Confirm that the HTO have the expected barcode names.

```
rownames(dataset_barcodes2)
```

6. Create a new assay to store barcode information and add this assay to the previously created Seurat object.

```
dataset_barcode_assay <-
 CreateAssayObject(counts =
 dataset_barcodes2)
 dataset[["barcodes"]] <-
 dataset_barcode_assay
```

7. Validate that the object now contains multiple assays.

```
DefaultAssay(dataset)
```

8. Normalize barcode data and perform demultiplexing via the HTODemux function. This method is described in detail in the following Seurat vignette: https://satijalab.org/seurat/articles/hashing_vignette

```
dataset <- HTODemux(dataset, assay
 = "barcodes", positive.quantile =
 0.99)
```

9. Group cells based on global classification results and remove cells with no barcode classification.

```
Idents(dataset) <-
 "barcodes_classification.global"
 dataset <- subset(dataset, idents =
 "Negative", invert = TRUE)
```

10. Group cells based on the maximum HTO signal.

```
Idents(dataset) <- "barcodes_maxID"
```

11. Visualize the distribution of detected genes in cells per their multiplexed barcodes (**Supplementary Figure 1**).

```
VlnPlot(dataset, features =
 "nFeature_RNA", pt.size = 0.1, log =
 TRUE)
```

12. Rename the barcodes to their actual wound time (days post-wounding) and space (2-8 mm) assignments (taken from the original manuscript) and assign them to a new metadata variable called time_space.

```
Idents(dataset) <- "barcodes_maxID"
 levels(dataset)
 dataset <- RenameIdents(dataset,
 "Barcode-1" ="D01_2mm",
 "Barcode-2" ="D01_4mm",
 "Barcode-3" ="D01_6mm",
 "Barcode-4" ="D01_8mm",
 "Barcode-5" ="D03_2mm",
 "Barcode-6" ="D03_4mm",
 "Barcode-7" ="D03_6mm",
 "Barcode-8" ="D03_8mm",
 "Barcode-9" ="D07_2mm",
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
"Barcode-10" ="D07_4mm",
"Barcode-11" ="D07_6mm",
"Barcode-12" ="D07_8mm",
"Barcode-13" ="D14_2mm",
"Barcode-14" ="D14_4mm",
"Barcode-15" ="D14_6mm",
"Barcode-16" ="D14_8mm",
"Barcode-17" ="UW")
levels(dataset)
dataset[["time_space"]] <-
Idents(dataset)
```

7. For datasets without multiplexing data: Create a Seurat object, while immediately filtering out genes expressed in less than 5 cells and cells with less than 200 genes detected.

```
dataset <- CreateSeuratObject(counts =
 b1, min.cells = 5, min.features = 200)
```

8. Switch to analyzing the gene expression assay of the dataset.

```
DefaultAssay(dataset) <- "RNA"
 Idents(dataset) <- "data"
```

9. As an important quality control step, calculate the percentage of mitochondrial genes in each cell and assign it as a metadata variable. This method is described in detail in the following Seurat vignette: https://satijalab.org/seurat/articles/pbmc3k_tutorial#qc-and-selecting-cells-for-further-analysis

```
dataset[["percent.mt"]] <-
 PercentageFeatureSet(dataset, pattern =
 "^mt-")
```

10. Visualize the distribution of detected genes, numbers of RNA and the mitochondrial percentage in all cells (**Supplementary Figure 2**).

```
plot1 <- FeatureScatter(dataset, feature1
 = "nCount_RNA", feature2 = "percent.mt")
 plot2 <- FeatureScatter(dataset,
 feature1 = "nFeature_RNA", feature2 = "
 percent.mt ")
 plot1 + plot2
```

11. There are a number of cells with large mitochondrial content, which correlates with low RNA counts; these are dead or dying cells. Remove these low-quality cells from the dataset using a fair cutoff. In this case, use the values of the original dataset described in the previously published study[20], in which cells with more than 25% mitochondrial genes are removed.

```
dataset <- subset(dataset, subset =
 nFeature_RNA > 200 & percent.mt < 25)
```

12. Visualize the distribution of detected genes, numbers of RNA, and the mitochondrial percentage in all cells after removing low-quality cells (**Supplementary Figure 3**).

```
plot1 <- FeatureScatter(dataset, feature1
 = "nCount_RNA", feature2 = "percent.mt")
 plot2 <- FeatureScatter(dataset,
 feature1 = "nFeature_RNA", feature2 = "
 percent.mt ")
 plot1 + plot2
```

13. As an additional important quality control step, detect likely doublets in the dataset. These are cells that were joined during droplet sequencing and will thus result in gene expressions that are not at the single-cell

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

level. To counter this problem, several tools have been developed. It is important to note that each tool makes certain assumptions about single-cell data, therefore it is important for the user to read all relevant documentation before using any one tool. In this workflow, implement a method called scDblFinder[23]. Please note that this method uses an algorithm that imposes a fixed expected doublet rate. Use the following commands to run the scDblFinder pipeline.

```
DefaultAssay(dataset) <- "RNA"
 sce <- scDblFinder(GetAssayData(dataset,
 assay="RNA", slot="counts"),
 samples=Idents(dataset))
```

NOTE: Multiplexed single-cell datasets such as this one may also be screened for doublets by removing cells that express multiple barcodes. This workflow did not demonstrate this method because most single-cell datasets do not have this unique feature. Instead, a more generalizable pipeline for screening for doublets is shown using the scDblFinder method.

14. Assign the doublet score to a new metadata variable.

```
dataset$scDblFinder.score <- sce
$scDblFinder.score
```

15. Visualize the distribution of the doublet score in all cells (**Supplementary Figure 4**).

```
VlnPlot(dataset, features =
 "scDblFinder.score", raster=FALSE,
 pt.size=0.5)
```

16. Remove the cells above the 0.25 double score threshold. This threshold was chosen based on the violin plot generated above, which showed that most cells in the dataset could be assigned to either very high or very low double scores, and 0.25 is a reasonable cutoff for this dataset that would remove the vast majority of likely doublets without removing many unlikely doublets.

```
dataset <- subset(dataset,
 scDblFinder.score < 0.25)
```

17. Save the dataset Seurat object as an RDS file into the working directory.

```
saveRDS(dataset,
 "dataset_post_Method2.rds")
```

3. Analyzing a single-cell wound healing dataset using Seurat

NOTE: (Optional step) If starting the workflow here, load in the saved RDS file as a Seurat object.

```
dataset <-
 readRDS("dataset_post_Method2.rds")
```

1. Perform the standard Seurat workflow for single-cell dataset normalization, scaling, and Principal Component Analysis (PCA). This standard workflow is described in the following Seurat vignettes:

   Seurat - Guided Clustering Tutorial: https://satijalab.org/seurat/articles/pbmc3k_tutorial

   Seurat Command List: https://satijalab.org/seurat/articles/essential_commands

   DefaultAssay(dataset) <- "RNA"

```
dataset <- NormalizeData(object =
 dataset)
 dataset <- FindVariableFeatures(object =
 dataset)
 dataset <- ScaleData(object = dataset)
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
dataset <- RunPCA(object = dataset)
```

2. Visualize the amount of dataset variation with respect to the first 50 PCA dimensions (**Supplementary Figure 5**).

```
ElbowPlot(dataset, reduction = "pca",
 ndims = 50)
```

Much of the major variation occurs within the first 13 dimensions.

3. Perform cell clustering of the dataset using a set PCA dimension range of 1-13 and a set resolution of 0.1.

```
dataset <- FindNeighbors(dataset, verbose
 = TRUE, dims = 1:13)
 dataset <- FindClusters(dataset, verbose
 = TRUE, resolution = 0.1)
```

NOTE: This workflow focuses on large scale differences in cell types. Therefore, it uses a fairly conservative parameter for the PCA dimension range, wherein the first 13 dimensions are shown to represent the vast majority of variation within the dataset. For discrimination of cells into smaller and rarer subtypes, the user can use a higher number of dimensions for downstream analysis since those rare cell subtypes likely account for lower levels of dataset variation. The resolution parameter ranges from 0 to 1, and it determines the magnitude of categorical separation imposed on the dataset. The setting of this parameter is dependent on the user's research question. For clustering of cells into numerous small and rare subtypes, use higher resolutions for downstream analysis. Since this workflow aims to explore more broad differences between major cell types, it uses a fairly small resolution value of 0.1, which is expected to cluster cells into fewer, larger groups. Use of the settings mentioned in step 3.3 will discriminate 8 unique cell clusters. These clusters are automatically assigned to a metadata variable called "seurat_clusters".

4. Perform UMAP dimensional reduction and finding neighbors analysis using the first 13 PCA dimensions. Add the seed number 123 to ensure the reproducibility of the resulting data projection.

```
dataset <- RunUMAP(dataset, verbose =
 TRUE, dims = 1:13, seed.use = 123)
```

NOTE: The UMAP algorithm is stochastic and introduces randomness into the dimensional reduction (see "Stability and Reproducibility" in https://cran.r-project.org/web/packages/umap/vignettes/umap.html). While using a consistent seed provides a "minimal level of reproducibility" to the algorithm, the resulting plot may still be slightly different from what is shown in the representative figures and downstream results. Testing has found that results will especially differ between computers running Windows and those running MacOS, likely due to different implementations of randomness in these operating systems.

5. Visualize the clustering of the cells on a UMAP plot (**Figure 1**).

```
DimPlot(dataset, group.by =
 "seurat_clusters", raster = FALSE, label
 = TRUE)
```

NOTE: The randomness of the UMAP algorithm may generate slightly different plots, as shown in the figure showing alternate UMAP plots generated using the same code as above on computers running Windows and MacOS; note the slight differences in the shapes of the

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

clusters. It is, therefore, imperative that the user saves and time-stamps all data and plots as they are generated, and that all downstream analyses on clusters be made carefully and with biological understanding in mind, as it is described below for cell type annotation.

6. Because the original experiment labels that refer to where and when the cells came from during wound healing are included, visualize the wound time/space annotation of the cells on a UMAP plot (**Figure 2**).

```
DimPlot(dataset, group.by = "time_space",
 raster = FALSE, label = FALSE)
```

7. Generate a table of cell cluster association with the wound time/space annotation.

```
table(dataset$time_space, dataset
$seurat_clusters)
```

8. Determine the identities of the major cell types in the dataset. To do this, calculate the differentially expressed genes (DEG) between all clusters. Get DEG lists for clusters, assign them to a variable, and save the output as a delimited text file within the working directory.

NOTE: This step is CPU intensive and can take a long time, depending on the user's hardware.

```
Idents(dataset) <- "seurat_clusters"
 Cell_markers <- FindAllMarkers(dataset,
 max.cells.per.ident = 500, only.pos =
 TRUE, min.pct = 0.10, logfc.threshold =
 0.25)
 write.csv(Cell_markers,
 file = file.path(getwd(),
 "dataset_cluster_markers.txt"))
```

9. Download and open the included **dataset_cluster_markers.txt** file in a spreadsheet (e.g., Excel) by copying the contents of the text file and using the **Text Import Wizard** to specify the comma delimiter and the identity of the gene name columns as **Text**. Indicating that the gene names are '**Text**' is important, otherwise Excel will automatically convert certain gene names to dates, e.g. Sept7 changing to September 7..

10. In a spreadsheet, filter the results according to the following recommended parameters:

    1. Rank the **avg_log2FC** column from largest to smallest in order to arrange all rows according to decreasing $\log_2$ fold changes (log2FC).

    2. Rank the **cluster** column from smallest to largest to arrange all rows according to increasing Seurat cluster numbers.

    3. Filter the **avg_log2FC** column for numbers greater than or equal to 2.5 to show only the most differentially expressed genes (DEGs) in the indicated cluster versus other clusters.

    4. Filter the **pct.1** column for numbers greater than or equal to 0.4. This column refers to percentage of cells in the indicated cluster expressing the indicated gene (Cluster %) and setting the threshold at 0.4 means that only genes expressed in at least 40% of the cells in the indicated cluster are shown.

    5. Filter the **pct.2** column for numbers less than or equal to 0.2. This column refers to the percentage of cells NOT in the indicated cluster expressing the indicated gene (Non-cluster %), and setting the threshold at 0.2 means that only genes expressed in at most 20% of the cells NOT in the indicated cluster are shown.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

6. Filter the **p_val_adj** column for numbers less than or equal to 0.01. This column refers to the adjusted P value or the false discovery rate (FDR), indicating the statistical strength of the indicated DEG, and setting the threshold at 0.01 means that only genes with an FDR < 0.01 are shown.

   NOTE: **Supplementary Table 1** (JoVE_DEGs_cellMarkers.xlsx) contains the full output of ranked differentially-expressed genes used in step 3.10 of the protocol. **Supplementary Table 2** shows the top 5 genes for each cluster in this analysis, with the bolded genes used for subsequent visualizations.

11. For unbiased cell-type annotation of clusters, use the EnrichR web-based enrichment analysis tool.
    Use the link: https://maayanlab.cloud/Enrichr/

12. Copy the lists of DEGs for each cluster into a separate EnrichR window, then click Analyze. The EnrichR tool runs the gene list through hundreds of curated databases and ranks every enriched term in each category.

13. For the purposes of cell type annotation, click the Cell Types tab above and focus on the top 5 enrichments in the three cell marker curated databases on the left-hand side (**Figure 3**):
    CellMarker 2024 (http://bio-bigdata.hrbmu.edu.cn/CellMarker/)
    Tabula Sapiens (https://tabula-sapiens-portal.ds.czbiohub.org/)
    PanglaoDB Augmented (https://panglaodb.se/)

14. Based on the enrichments of the DEGs in these databases, confirm the likely identity of the 8 clusters. Note that there are two clusters (2, 6) that enrich as fibroblasts; therefore, combine these clusters into single cell type annotations. Assign the cell type identities as labels to a new metadata variable called cell_types.

```
Idents(dataset) <- "seurat_clusters"
 dataset[["cell_types"]] <-
 Idents(dataset)
 Idents(dataset) <- "cell_types"
 dataset <- RenameIdents(dataset,
 "0" = "Macrophage",
 "1" = "Neutrophil",
 "2" = "Fibroblast",
 "3" = "Epithelial cell",
 "4" = "Endothelial cell",
 "5" = "T cell",
 "6" = "Fibroblast",
 "7" = "Smooth muscle cell"
 )
 levels(dataset)
 dataset[["cell_types"]] <-
 Idents(dataset)
```

15. Visualize the renamed cell clusters as annotations on a UMAP plot (**Figure 4**).

```
DimPlot(dataset, group.by = "cell_types",
 raster = FALSE, label = FALSE)
```

16. Visualize the localization of the top (bolded) cluster marker genes from Table 1 on a series of UMAP plots (**Figure 5**).

```
DefaultAssay(dataset) <- "RNA"
 FeaturePlot(dataset, features =
 c("Arg1", "Retnlg", "Fgf7", "Dsp",
 "Tie1", "Cd3g", "Cdh4", "Rgs5"),
 raster=FALSE, ncol = 4)
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

17. Visualize the top cluster marker DEGs on a dot plot, grouped by the original cluster numbers (**Supplementary Figure 6**).

```
DotPlot(dataset, group.by =
 "seurat_clusters", features =
 c("Arg1","Retnlg","Fgf7","Dsp","Tie1",
 "Cd3g","Cdh4","Rgs5")) + RotatedAxis() +
 scale_colour_gradient2(low = "blue", mid
 = "grey", high = "red")
```

18. Visualize the top cluster marker DEGs on a dot plot, grouped by the annotated cell types (**Figure 6**).

```
DotPlot(dataset, group.by
 = "cell_types", features =
 c("Arg1","Retnlg","Fgf7","Dsp","Tie1",
 "Cd3g","Cdh4","Rgs5")) + RotatedAxis() +
 scale_colour_gradient2(low = "blue", mid
 = "grey", high = "red")
```

19. To perform time-series analyses, first simplify the dataset to remove the spatial component. For time-course analyses, group the wound time/space annotations into overall days post-wounding (DPW) with a new metadata variable called "DPW".

```
Idents(dataset) <- "time_space"
 dataset[["DPW"]] <- Idents(dataset)
 Idents(dataset) <- "DPW"
 new.cluster.ids <- c("D1", "D1", "D1",
 "D1", "D3", "D3", "D3", "D3", "D7",
 "D7", "D7", "D7", "D14", "D14", "D14",
 "D14", "UW")
 names(new.cluster.ids) <-
 levels(dataset)
 dataset <- RenameIdents(dataset,
 new.cluster.ids)
 dataset[["DPW"]] <- Idents(dataset)
 Idents(dataset) <- "DPW"
```

20. Visualize the new wound time-course groupings on a UMAP plot (**Supplementary Figure 7**).

```
DimPlot(dataset, group.by = "DPW", raster
 = FALSE, label = FALSE)
```

21. Generate tables showing how many cells of each type occur in each DPW.

```
table(dataset$DPW, dataset$cell_types)
```

    1. Optional Step: To also get DEG lists for wound time-course groups, assign them to a variable, and save the output as a delimited text file.

```
Idents(dataset) <- "DPW"
 Cell_DPW_markers <-
 FindAllMarkers(dataset,
 max.cells.per.ident = 500,
 only.pos = TRUE, min.pct = 0.10,
 logfc.threshold = 0.25)
 write.csv(Cell_DPW_markers,
 file = file.path(getwd(),
 "dataset_DPW_markers.txt"))
```

22. Convert the cell numbers into proportions per category to better understand the relative changes in cell type composition throughout the time course of healing. Visualize the proportion of DPW in each cell type (**Supplementary Figure 8**):

```
pt1 <- table(dataset$DPW, dataset
$cell_types)
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
pt1 <- as.data.frame(pt1)
pt1$Var1 <- as.character(pt1$Var1)
ggplot(pt1, aes(x = Var2, y = Freq, fill
= Var1)) +
theme_bw(base_size = 15) +
geom_col(position = "fill", width = 0.5)
+
xlab("Sample") + RotatedAxis() +
ylab("Proportion") +
theme(legend.title = element_blank())
```

23. Visualize the proportion of cell types in each DPW (**Figure 7**).

```
pt2 <- table(dataset$cell_types, dataset
$DPW)
pt2 <- as.data.frame(pt2)
pt2$Var1 <- as.character(pt2$Var1)
ggplot(pt2, aes(x = Var2, y = Freq, fill
= Var1)) +
theme_bw(base_size = 15) +
geom_col(position = "fill", width = 0.5)
+
xlab("Sample") +
ylab("Proportion") +
theme(legend.title = element_blank())
```

24. Save the dataset Seurat object as an RDS file into the working directory.

```
saveRDS(dataset,
"dataset_post_Method3.rds")
```

4. Analyzing cell subtypes using Seurat

NOTE: The power of single-cell analysis enables the discovery and analysis of rare subtypes within the major cell types that are analyzed above. This example focuses on fibroblasts, which initially clustered into two Seurat clusters before they were combined into a single category. This part of the protocol focuses on fibroblasts specifically, excluding all other cell types, and explores their identities and temporal properties during wound healing. As an optional step, if needed, load the saved RDS file as a Seurat object.

```
dataset <-
readRDS("dataset_post_Method3.rds")
```

1. Subset the original dataset according to the fibroblast cell identity.

   Idents(dataset) <- "cell_types"

```
dataset_fibroblast <- subset(dataset,
idents = "Fibroblast")
```

2. Perform PCA on this smaller dataset and visualize the amount of dataset variation with respect to PCA dimensions (**Supplementary Figure 9**).

```
dataset_fibroblast <-
RunPCA(dataset_fibroblast, verbose =
TRUE)
ElbowPlot(dataset_fibroblast, reduction
= "pca", ndims = 50)
```

   NOTE: Much of the major variation occurs within the first 9 dimensions.

3. Perform cell clustering of the dataset using a set PCA dimension range of 1-9 and a set resolution of 0.1.

```
dataset_fibroblast <-
FindNeighbors(dataset_fibroblast,
verbose = TRUE, dims = 1:9)
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
dataset_fibroblast <-
FindClusters(dataset_fibroblast, verbose
= TRUE, resolution = 0.1)
```

NOTE: Using these settings, the algorithm discriminates 3 unique fibroblast clusters. These clusters are automatically assigned to a metadata variable called seurat_clusters.

4. Perform UMAP dimensional reduction and finding neighbors analysis using the first 9 PCA dimensions. Add the seed number 123 to ensure the reproducibility of the resulting data projection.

```
dataset_fibroblast <-
 RunUMAP(dataset_fibroblast, verbose =
 TRUE, dims = 1:9, seed.use = 123)
```

5. Visualize the clustering of the cells on a UMAP plot (**Figure 8**).

```
DimPlot(dataset_fibroblast, group.by =
 "seurat_clusters", raster = FALSE, label
 = TRUE)
```

6. Visualize the wound time-course annotation of the cells on a UMAP plot (**Supplementary Figure 10**).

```
DimPlot(dataset_fibroblast, group.by =
 "DPW", raster = FALSE, label = FALSE)
```

7. Get DEG lists for the three fibroblast subtypes and save them into a text file in the working directory.

```
Idents(dataset_fibroblast) <-
 "seurat_clusters"
 fibroblast_markers <-
 FindAllMarkers(dataset_fibroblast,
 max.cells.per.ident = 500, only.pos =
 TRUE, min.pct = 0.10, logfc.threshold =
 0.25)
 write.csv(fibroblast_markers,
 file = file.path(getwd(),
 "fibroblast_cluster_markers.txt"))
```

8. Follow similar steps as above (steps 3.9-3.10) in a spreadsheet to filter the DEGs into the top fibroblast subtype markers.

9. Define a custom gene list as a variable by copying the top 5 fibroblast subtype markers for each of the three clusters from the DEG text file.

```
FB_type_marker <- c("Plac8", "Cthrc1",
 "Adam12", "Ifi211", "Plat", "Cdh4",
 "Aldh3a1", "Ppp1r14a", "Oxtr",
 "Serpina3c", "Sostdc1", "Scube3",
 "Ptprz1", "Corin", "Alx4")
```

10. Visualize the genes in the list in the fibroblast-only dataset by calling the variable in the features parameter of the dotplot (**Figure 9**).

```
DotPlot(dataset_fibroblast,
 group.by="seurat_clusters", features
 = FB_type_marker) + RotatedAxis() +
 scale_colour_gradient2(midpoint = 0, low
 = "blue", mid = "grey", high = "red")
 DotPlot(dataset_fibroblast,
 group.by="DPW", features =
 FB_type_marker) + RotatedAxis() +
 scale_colour_gradient2(midpoint = 0, low
 = "blue", mid = "grey", high = "red")
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

11. Visualize the genes in the list in the original single-cell dataset by calling the variable in the features parameter of the dotplot (**Supplementary Figure 11**).

```
DotPlot(dataset, group.by="cell_types",
 features = FB_type_marker)
 + RotatedAxis() +
 scale_colour_gradient2(midpoint = 0, low
 = "blue", mid = "grey", high = "red")
```

12. Visualize the proportion of fibroblast subtypes in each DPW category (**Supplementary Figure 12**).

```
pt3 <- table(dataset_fibroblast
$seurat_clusters, dataset_fibroblast$DPW)
 pt3 <- as.data.frame(pt3)
 pt3$Var1 <- as.character(pt3$Var1)
 ggplot(pt3, aes(x = Var2, y = Freq, fill
 = Var1)) +
 theme_bw(base_size = 15) +
 geom_col(position = "fill", width = 0.5)
 +
 xlab("Sample") +
 ylab("Proportion") +
 theme(legend.title = element_blank())
```

13. Visualize the proportion of DPW fibroblast cells in each fibroblast subtype category (**Supplementary Figure 13**).

```
pt4 <- table(dataset_fibroblast$DPW,
 dataset_fibroblast$seurat_clusters)
 pt4 <- as.data.frame(pt4)
 pt4$Var1 <- as.character(pt4$Var1)
 ggplot(pt4, aes(x = Var2, y = Freq, fill
 = Var1)) +
 theme_bw(base_size = 15) +
 geom_col(position = "fill", width = 0.5)
 +
 xlab("Sample") +
 ylab("Proportion") +
 theme(legend.title = element_blank())
```

14. Save the dataset Seurat object as an RDS file into the working directory.

```
saveRDS(dataset_fibroblast,
 "dataset_fibroblast_post_Method4.rds")
```

5. Example of a follow-up analysis via module scoring

NOTE: One useful method for analyzing single-cell datasets is called module scoring. In this workflow, one can define a gene list according to previous knowledge and then calculate module scores, which can identify potential enrichments of the gene list within each cell. These scores can be averaged across cell annotations to reveal potential patterns of enrichments.

Here, use gene lists from a previously published study[2], where wound healing phase-specific genes were identified using bulk RNA-sequenced samples from across the healing continuum. The gene lists were saved into a tab-delimited text file (**Supplementary File 2: JoVE_PhaseSpecificGenes.txt**) that can now be downloaded into the working directory and used to generate gene lists that identify the three major healing phases.

1. Load in gene lists into a variable by reading the TEXT file.

```
PhaseSpecificGenes <-
 read_delim("JoVE_PhaseSpecificGenes.txt",
 delim="\t", col_names = T)
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

2. Separate the columns into individual gene list variables and change the genes to mouse names that have their first letter capitalized.

```
PS_Inflammatory <- PhaseSpecificGenes[1]
 PS_Inflammatory_ms <-
 lapply(PS_Inflammatory, str_to_sentence)
 PS_Proliferative <-
 PhaseSpecificGenes[2]
 PS_Proliferative_ms <-
 lapply(PS_Proliferative,
 str_to_sentence)
 PS_Resolution <- PhaseSpecificGenes[3]
 PS_Resolution_ms <-
 lapply(PS_Resolution, str_to_sentence)
```

NOTE: (Optional) If needed, load the saved RDS file as a Seurat object.

```
dataset <-
 readRDS("dataset_post_Method3.rds")
```

3. Use the gene lists as modules to score each cell in the dataset according to the three phases of healing.

```
dataset <- AddModuleScore(
 object = dataset,
 features = PS_Inflammatory_ms,
 ctrl = 100,
 name = 'Inflammatory'
 )
 dataset <- AddModuleScore(
 object = dataset,
 features = PS_Proliferative_ms,
 ctrl = 100,
 name = 'Proliferative'
 dataset <- AddModuleScore(
 object = dataset,
 features = PS_Resolution_ms,
 ctrl = 100,
 name = 'Resolution'
 )
```

4. Visualize the aggregate module scores per cell category, including DPW and major cell types (**Figure 10**).

```
DotPlot(dataset, group.by="DPW", features
 = c("Inflammatory1","Proliferative1",
 "Resolution1")) + RotatedAxis() +
 scale_colour_gradient2(midpoint = 0, low
 = "blue", mid = "grey", high = "red")
 DotPlot(dataset,
 group.by="cell_types", features =
 c("Inflammatory1","Proliferative1",
 "Resolution1")) + RotatedAxis() +
 scale_colour_gradient2(midpoint = 0, low
 = "blue", mid = "grey", high = "red")
```

6. Example of a follow-up analysis via CellChat

NOTE: Another useful and well-cited method for the analysis of single-cell datasets is inferring cell-cell interactions. In this workflow, use the package CellChat, which infers cell-cell communications by analyzing differential ligand-receptor interactions between cell groups[22]. Recently, the developers of CellChat published a detailed step-by-step protocol for its generalized use[24], and this is an excellent resource for users as they work through the following workflow and apply it to their datasets. As an example, the following workflow compares interactions of all major cells in wounds at 1 vs 14 days post-wounding (DPW). Each step is not described in great detail, since all steps have already been described in

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

the official CellChat publication[24] as well as tutorials, linked here:

Inference and analysis of cell-cell communication using CellChat: https://github.com/jinworks/CellChat/blob/master/tutorial/CellChat-vignette.html

Comparison analysis of multiple datasets using CellChat: https://github.com/jinworks/CellChat/blob/master/tutorial/Comparison_analysis_of_multiple_datasets.html

Optional step: If needed, load in the saved RDS file as a Seurat object:

```
dataset <-
 readRDS("dataset_post_Method3.rds")
```

1. Subset the original dataset into two datasets according to the DPW annotation.

```
Idents(dataset) <- "DPW"
 dataset_D1 <- subset(dataset, ident =
 "D1")
 dataset_D14 <- subset(dataset, ident =
 "D14")
```

2. Define the annotation by which CellChat will be performed --- in this case, use the major cell types.

```
Idents(dataset_D1) <- "cell_types"
 Idents(dataset_D14) <- "cell_types"
```

3. Create the CellChat objects and follow the typical CellChat workflow. See tutorials linked above as detailed references for each step.

```
cellchat_D1 <- createCellChat(dataset_D1,
 group.by = "ident", assay = "RNA")
cellchat_D14 <-
 createCellChat(dataset_D14, group.by =
 "ident", assay = "RNA")
CellChatDB <- CellChatDB.mouse
CellChatDB.use <- CellChatDB
cellchat_D1@DB <- CellChatDB.use
cellchat_D14@DB <- CellChatDB.use
cellchat_D1 <- subsetData(cellchat_D1)
cellchat_D14 <- subsetData(cellchat_D14)
future::plan("multisession", workers =
4)
cellchat_D1 <-
 identifyOverExpressedGenes(cellchat_D1,
 do.fast = F)
cellchat_D14 <-
 identifyOverExpressedGenes(cellchat_D14,
 do.fast = F)
cellchat_D1 <-
identifyOverExpressedInteractions(cellchat_D1)
cellchat_D14 <-
identifyOverExpressedInteractions(cellchat_D14)
cellchat_D1 <-
 computeCommunProb(cellchat_D1, type =
 "triMean", population.size = TRUE)
cellchat_D14 <-
 computeCommunProb(cellchat_D14, type =
 "triMean", population.size = TRUE)
cellchat_D1 <-
 filterCommunication(cellchat_D1,
 min.cells = 10)
cellchat_D14 <-
 filterCommunication(cellchat_D14,
 min.cells = 10)
cellchat_D1 <-
 computeCommunProbPathway(cellchat_D1)
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
cellchat_D14 <-
computeCommunProbPathway(cellchat_D14)
cellchat_D1 <- aggregateNet(cellchat_D1)
cellchat_D14 <-
aggregateNet(cellchat_D14)
cellchat_D1 <-
netAnalysis_computeCentrality(cellchat_D1,
slot.name = "netP")
cellchat_D14 <-
netAnalysis_computeCentrality(cellchat_D14,
slot.name = "netP")
```

4. Visualize the incoming vs outgoing interaction strengths in all major cell types at each wound healing time point (**Supplementary Figure 14**).

```
netAnalysis_signalingRole_scatter(cellchat_D1)
netAnalysis_signalingRole_scatter(cellchat_D14)
```

Fibroblasts dramatically increase their interactions between D1 and D14 DPW.

5. Show the lists of all significant inferred cell-cell communication pathways.

```
cellchat_D1@netP$pathways
 cellchat_D14@netP$pathways
```

The collagen pathway is one of the significant pathways on both D1 and D14 DPW.

6. Focus on the collagen signaling pathway and its interaction with fibroblasts.

```
pathways.show <- c("COLLAGEN")
```

7. Visualize the collagen signaling pathway interactions between cell types using circle diagrams (**Supplementary Figure 15**).

```
par(mfrow=c(1,2))
 netVisual_aggregate(cellchat_D1,
 signaling = pathways.show, layout =
 "circle")
 netVisual_aggregate(cellchat_D14,
 signaling = pathways.show, layout =
 "circle")
 par(mfrow=c(1,1))
```

8. Visualize the collagen signaling pathway interactions between cell types using chord diagrams (**Supplementary Figure 16**).

```
par(mfrow=c(1,2))
 strwidth <- function(x) {0.5}
 netVisual_aggregate(cellchat_D1,
 signaling = pathways.show, layout =
 "chord", vertex.label.cex = 0.6)
 netVisual_aggregate(cellchat_D14,
 signaling = pathways.show, layout =
 "chord", vertex.label.cex = 0.6)
 par(mfrow=c(1,1))
```

9. Visualize the COLLAGEN signaling pathway interactions with fibroblasts as source cells (**Supplementary Figure 17**).

NOTE: The cell types in the cellchat objects are listed as IDs in the order that they were assigned in the original Seurat object: 1 = Macrophage, 2 = Neutrophil, 3 = Fibroblast, 4 = Epithelial cell, 5 = Endothelial cell, 6 = T cell, 7 = Smooth muscle cell.

```
par(mfrow=c(1,2))
 strwidth <- function(x) {0.5}
 netVisual_aggregate(cellchat_D1,
 signaling = pathways.show, layout
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
= "chord", vertex.label.cex = 0.6,
sources.use = 3)
netVisual_aggregate(cellchat_D14,
signaling = pathways.show, layout
= "chord", vertex.label.cex = 0.6,
sources.use = 3)
par(mfrow=c(1,1))
```

10. Visualize the contributions of each ligand-receptor pair in the COLLAGEN signaling pathway with fibroblasts as source cells.

    1. Using bubble plots (**Supplementary Figure 18**):

```
gg1 <- netVisual_bubble(cellchat_D1,
 sources.use = 3, targets.use =
 NULL, signaling = pathways.show,
 remove.isolate = FALSE)
 gg2 <-
 netVisual_bubble(cellchat_D14,
 sources.use = 3, targets.use =
 NULL, signaling = pathways.show,
 remove.isolate = FALSE)
 gg1 + gg2
```

    2. Using chord diagrams (**Supplementary Figure 19**):

```
par(mfrow=c(1,2))
 strwidth <- function(x) {0.4}
 netVisual_chord_gene(cellchat_D1,
 sources.use = 3, targets.use = NULL,
 signaling = pathways.show, lab.cex =
 0.6, show.legend= F)
 netVisual_chord_gene(cellchat_D14,
 sources.use = 3, targets.use = NULL,
 signaling = pathways.show, lab.cex =
 0.6, legend.pos.x = 60)
 par(mfrow=c(1,1))
```

11. Focus on the Col1a1-Cd44 ligand-receptor interaction within the COLLAGEN signaling pathway.

```
LR.show <- "COL1A1_CD44"
```

12. Visualize the Col1a1-Cd44 ligand-receptor interactions between cell types using chord diagrams (**Supplementary Figure 20**).

```
strwidth <- function(x) {0.5}
 netVisual_individual(cellchat_D1,
 signaling = pathways.show, pairLR.use =
 LR.show, layout = "chord")
 netVisual_individual(cellchat_D14,
 signaling = pathways.show, pairLR.use =
 LR.show, layout = "chord")
```

13. Perform differential CellChat analysis by generating a combined CellChat object.

```
object.list_D14_v_D1 <- list(D1 =
 cellchat_D1, D14 = cellchat_D14)
 cellchat_D14_v_D1 <-
 mergeCellChat(object.list_D14_v_D1,
 add.names = names(object.list_D14_v_D1))
```

14. Visualize the total numbers and relative strengths of cell-cell interactions in the wound healing time points (**Supplementary Figure 21**).

```
gg1 <-
 compareInteractions(cellchat_D14_v_D1,
 show.legend = F)
 gg2 <-
 compareInteractions(cellchat_D14_v_D1,
 show.legend = F, measure = "weight")
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
gg1 + gg2
```

15. Visualize using a circle plot the differential cell-cell interaction strengths between each cell type as the wound transitions from day 1 to day 14 (**Supplementary Figure 22**).

```
netVisual_diffInteraction(cellchat_D14_v_D1,
 weight.scale = T, measure = "weight")
```

16. Visualize using a heatmap the differential cell-cell interaction strengths between each cell type as the wound transitions from day 1 to day 14 (**Supplementary Figure 23**).

```
netVisual_heatmap(cellchat_D14_v_D1,
 measure = "weight")
```

17. Visualize using a rank plot the relative contributions of individual pathways to cell-cell interactions with fibroblasts as source cells at day 14 vs day 1 (Supplementary **Figure 24**).

```
rankNet(cellchat_D14_v_D1, mode =
 "comparison", measure = "weight",
 sources.use = 3, targets.use = NULL,
 stacked = T, do.stat = TRUE)
```

18. Visualize using bubble plots the relative contributions of individual ligand-receptor pairs in the collagen signaling pathway with fibroblasts as source cells at day 14 compared to day 1 (**Supplementary Figure 25**).

```
gg1 <-
 netVisual_bubble(cellchat_D14_v_D1,
 sources.use = 3, targets.use = NULL,
 signaling = pathways.show, comparison =
 c(1, 2), max.dataset = 2, title.name =
 "Increased signaling in D14", angle.x =
 45, remove.isolate = F)
 gg2 <-
 netVisual_bubble(cellchat_D14_v_D1,
 sources.use = 3, targets.use = NULL,
 signaling = pathways.show, comparison =
 c(1, 2), max.dataset = 1, title.name =
 "Decreased signaling in D14", angle.x =
 45, remove.isolate = F)
 gg1 + gg2
```

19. Just like Seurat objects, CellChat objects can be saved and opened as RDS files.

```
saveRDS(cellchat_D1, "cellchat_D1.rds")
 saveRDS(cellchat_D14,
 "cellchat_D14.rds")
 saveRDS(cellchat_D14_v_D1,
 "cellchat_D14_v_D1.rds")
```

20. Optional step: CellChat objects can also be opened from RDS files.

```
cellchat_D1 <- readRDS("cellchat_D1.rds")
 cellchat_D14 <-
 readRDS("cellchat_D14.rds")
 cellchat_D14_v_D1 <-
 readRDS("cellchat_D14_v_D1.rds")
```

7. Example of an integrative analysis by combining multiple single-cell datasets

NOTE: Single-cell datasets are often separated into multiple files because they were sequenced in batches or groups. This workflow shows how to integrate two of the five batches of the wound healing dataset [20]. The current methods for dataset integration are described by the following Seurat vignettes:

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

Introduction to scRNA-seq integration:

https://satijalab.org/seurat/articles/integration_introduction

Integrative analysis in Seurat v5:

https://satijalab.org/seurat/articles/seurat5_integration

Note: There are numerous single-cell dataset integration methods, each with its own strengths and weaknesses. For details, see the comprehensive benchmark of integration methods[25]. It is important for the user to read all relevant documentation before relying on any one integration method.

1. Repeat all steps in Method 2 for another batch of the Hu et al. dataset[20]. In the following protocol, batch #3 is used. The supplementary R script file is included and can be used to process batch #3 (**Supplementary File 3: JoVE_Rscript_b3.R**). Remember to create and use a new variable for the dataset --- in the code below, use "dataset_b3" for dataset batch #3.
    1. Optional Step: If needed, open the two datasets as Seurat objects from their RDS files saved in the working directory:

```
dataset <-
 readRDS("dataset_post_Method2.rds")
 dataset_b3 <-
 readRDS("dataset_b3_post_Method2.rds")
```

2. Assign a new variable to each dataset named "batch" to label the dataset of origin in subsequent analyses.

```
dataset@meta.data$batch <- "b1"
 dataset_b3@meta.data$batch <- "b3"
```

3. Perform Seurat merging of the two datasets, adding batch-based cell ID annotations, and then perform the standard Seurat workflow for the merged dataset, as described in Method 3.

```
dataset_merged <- merge(x = dataset, y
 = c(dataset_b3), add.cell.ids = c("b1",
 "b3"), merge.data = TRUE)
 DefaultAssay(dataset_merged) <- "RNA"
 dataset_merged <-
 NormalizeData(dataset_merged)
 dataset_merged <-
 FindVariableFeatures(dataset_merged)
 dataset_merged <-
 ScaleData(dataset_merged)
 dataset_merged <- RunPCA(dataset_merged)
 ElbowPlot(dataset_merged, reduction =
 "pca", ndims = 50)
```

4. Perform clustering and UMAP analysis on the combined dataset prior to data integration.

```
dataset_merged <-
 FindNeighbors(dataset_merged, dims =
 1:10, reduction = "pca")
 dataset_merged <-
 FindClusters(dataset_merged,
 resolution = .1, cluster.name =
 "unintegrated_clusters")
 dataset_merged <-
 RunUMAP(dataset_merged, dims = 1:10,
 seed.use = 123, reduction = "pca",
 reduction.name = "umap.unintegrated")
```

5. Visualize the UMAP plot according to cluster and batch numbers (**Supplementary Figure 26**).

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

```
DimPlot(dataset_merged, reduction
 = "umap.unintegrated", group.by =
 c("seurat_clusters", "batch"))
```

6. Show the distribution of cell numbers in each cluster according to batch number.

```
table(dataset_merged$batch,
 dataset_merged$seurat_clusters)
```

NOTE: From the UMAP plot and the table, there does not appear to be any significant batch effects for these two datasets. Evidence of batch effects would be manifested as unexpected discrepancies in cluster distribution between the two datasets, which could mean that there are potential technical differences between the datasets that are overriding actual biological similarities.

7. Perform Seurat data integration using the RPCA method. For more information on this and other data integration methods, please read the Seurat vignette linked above.

```
dataset_merged <- IntegrateLayers(
 object = dataset_merged, method =
 RPCAIntegration,
 orig.reduction = "pca", new.reduction =
 "integrated.rpca",
 verbose = TRUE
 )
```

8. Perform clustering and UMAP analysis on the combined dataset after data integration.

```
FindNeighbors(dataset_merged, dims =
 1:10, reduction = "integrated.rpca")
 dataset_merged <-
 FindClusters(dataset_merged, resolution
 = .1, cluster.name = "rpca_clusters")
 dataset_merged <-
 RunUMAP(dataset_merged, dims =
 1:10, seed.use = 123, reduction =
 "integrated.rpca", reduction.name =
 "umap.rpca")
```

9. Visualize the UMAP plot according to cluster and batch numbers after integration (**Supplementary Figure 27**).

```
DimPlot(dataset_merged, reduction
 = "umap.rpca", group.by =
 c("seurat_clusters","batch"))
```

10. Show distribution of cell numbers in each cluster according to batch number after integration.

```
table(dataset_merged$batch,
 dataset_merged$seurat_clusters)
```

From the UMAP plot and the table of the integrated data, there is now excellent overlap between the two batches across different clusters. Interestingly, the integration of the data resulted in the identification of an additional cluster using the same clustering parameters.

11. After dataset integration and prior to downstream analyses, the layers of the merged dataset must be joined.

```
dataset_merged <-
 JoinLayers(dataset_merged)
```

12. Save the dataset **Seurat object** as an RDS file into the working directory.

```
saveRDS(dataset_merged,
 "merged_dataset_post_Method7.rds")
```

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

Representative Results

Beginning in method #2, the protocol walks through the steps for loading in and performing quality control steps on a single-cell wound healing dataset. After creating the Seurat object (step 2.6.2), a series of steps merges the two assays within the dataset (RNA and protein; steps 2.6.3-2.6.7) and performs decomplexing of the protein assay according to spatio-temporal barcodes (steps 2.6.8-2.6.9). The decomplexing function assigns several metadata labels to each cell in the dataset, including "barcodes_maxID", which identifies the most probable spatio-temporal barcode of each cell (step 2.6.10). In step 2.6.11, the violin plot function is performed to visualize the distribution of detected genes in cells based on their multiplexed barcodes. The representative result of this step (**Supplementary Figure 1**) shows that there is a fairly even distribution of detected genes for each barcode, which is important for dataset integrity and downstream analysis of wound healing time points. After assigning the appropriate label to the protein barcodes (step 2.6.12), the protocol then shows how to perform quality control steps on the RNA assay of the dataset, beginning with calculating the percentage of mitochondrial genes in each cell (step 2.9). In step 2.10, the feature scatter plot function is performed to visualize the distribution of detected genes, the number of RNA, and the mitochondrial percentage in all cells. The representative results of this step (**Supplementary Figure 2**) show that there are a number of cells with large mitochondrial content, which correlates with low RNA counts and identifies dead or dying cells. After removing cells with low RNA counts and large mitochondrial contents (step 2.11), in step 2.12 another feature scatter plot function is performed on the subset dataset, and the representative result of this step (**Supplementary Figure 3**) shows that the distribution of detected genes and percentage mitochondrial RNA per cell is now more normal, clearing the way for robust downstream analyses. Next, the protocol describes the use of the scDblFinder function to identify likely doublets in the dataset and assigns a new metadata called "scDblFinder.score" to each cell (steps 2.13-2.14). In step 2.15, the violin plot function is performed to visualize the distribution of doublet scores in the dataset, and the representative result of this step (**Supplementary Figure 4**) shows that there are a number of cells with relatively high doublet scores, and that 0.25 looks to be a natural cutoff above which there is a population of likely doublets. Therefore, the following steps use this parameter to subset the dataset to cells below the cutoff (step 2.16), thereby completing the quality control steps for this single-cell dataset.

Beginning in method #3, the protocol walks through the steps for analyzing the quality-controlled single-cell wound healing dataset using the Seurat package and workflow. After normalization and scaling of the RNA data, PCA analysis is performed (step 3.1). In step 3.2, the elbow plot function is used to visualize the amount of dataset variation with respect to the first 50 PCA dimensions, and the representative result of this step (**Supplementary Figure 5**) shows that much of the major variation occurs within the first 13 dimensions as identified by the bend in the graph. The protocol then shows how to find neighbors and perform cell clustering (step 3.3) and UMAP dimensional reduction (step 3.4) of the dataset using the first 13 PCA dimensions and a relatively low clustering resolution parameter of 0.1, both of which were chosen in order to identify the most generalizeable major cell types in wounds. In step 3.5, the dimensional plot function is performed to visualize the clustering of the cells on a UMAP plot, and the representative result of this step (**Figure 1**) shows that all cells in the

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

dataset are clustered around 8 major color-coded Seurat cluster groups, with slightly different UMAP plots obtained from a computer running Windows (left) and MacOS (right). In step 3.6, another dimensional plot function is performed to visualize the wound time/space annotation of the cells, and the representative result of this step (**Figure 2**) shows that all cells in the dataset are spread out according to their time/space origin, with no apparent clustering according to the time/space annotation. The protocol next describes how to obtain lists of differentially-expressed genes and save them into a text file (step 3.8), open the data table in a spreadsheet, and perform various filtering steps in order to obtain the top ranked cluster markers for each cell cluster (steps 3.9-3.10.6). The representative result of these steps (**Supplementary Table 1**) is the final spreadsheet file containing the full output of ranked differentially-expressed genes, while another representative result (**Supplementary Table 2**) is a simplified table showing the top 5 up-regulated and expressed genes for each Seurat cluster. The protocol then describes how to use a web-based functional enrichment analysis tool called EnrichR to identify putative cell types according to the top cluster marker genes (steps 3.11-3.12), and the representative results of these steps (**Figure 3**) are cropped screenshots of the EnrichR outputs showing the top enriched cell types for each of the eight cell clusters. The protocol then assigns a new metadata label called "cell_types" to all of the cells in the respective Seurat clusters according to their most enriched cell type annotations (step 3.14). In step 3.15, the dimensional plot function is performed to visualize the renamed cell clusters as cell type annotations on a UMAP plot, and the representative results of this step (**Figure 4**) showed that all cells in the dataset clustered around the major color-coded cell types. In step 3.16, the feature plot function was used to visualize the localization of the top cluster marker genes (from Supplementary Table 2) on a series of UMAP plots, and the representative results (**Figure 5**) are a grid of UMAP plots showing the high expression of the top cell marker genes within their respective major cell type cluster locations. In steps 3.17 and 3.18, the dot plot function was performed to visualize the relative expression levels of top cluster marker genes in cells, first grouped by their original Seurat cluster numbers (step 3.17) and secondly grouped by annotated cell type labels (step 3.18). The representative results of these steps confirmed the high level of expression of the top cell marker genes only in their respective Seurat clusters (**Supplementary Figure 6**) and only in their respective major cell types (**Figure 6**). The next step in the protocol simplifies the original spatial-temporal protein-based labels into strictly temporal annotations, which identify cells based on the days post-wounding (DPW) from which they originated. In step 3.20, the dimensional plot function is performed to visualize the cells as DPW annotations on a UMAP plot, and the representative results of this step (**Supplementary Figure 7**) showed the localization of wound time-course annotations across the single-cell wound healing dataset. As expected, day 1 (D1) annotations dominated the neutrophil and macrophage clusters, whereas the later wound healing time points were more represented in other cell types. The following steps in the protocol used stacked bar plots to first visualize the proportions of DPW across different cell types (step 3.22) and then to visualize the proportions of cell types across different time points (3.23). The representative results of these steps are proportion plots showing the relative numbers of DPW cells in each major cell type category (**Supplementary Figure 8**) and the relative numbers of major cell types in each DPW category (**Figure 7**). These results confirmed the known cellular cascade of skin wound healing, wherein immune cells (neutrophils and macrophages) dominate the early time points during the inflammatory phase, and the

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

other cell types (epithelial cells and endothelial cells) begin to appear during the proliferative phase, with fibroblasts being especially dominant in the later time points during wound resolution.

Beginning in method #4, the protocol outlines the steps for using Seurat to focus on an individual major cell type in the single-cell dataset in order to identify potential cellular subtypes during wound healing. The protocol focuses on fibroblasts, which initially clustered into two Seurat clusters before they were combined into a single category, and describes how to create a new Seurat object that only contains the fibroblasts from the original dataset (step 4.1). The Seurat workflow is performed on this fibroblast-specific dataset (steps 4.2-4.4), with step 4.2 resulting in an elbow plot (**Supplementary Figure 9**) showing that much of the major variation in the fibroblast dataset occurs within the first 9 PCA dimensions. In step 4.5, the dimensional plot function is performed to visualize the clustering of the cells on a UMAP plot, and the representative results of this step (**Figure 8**) showed the fibroblasts in the dataset clustered around the 3 color coded cell subtypes. Visualizing the fibroblast dataset according to their DPW annotation (step 4.6), resulted in a UMAP plot (**Supplementary Figure 10**) showing fibroblasts in the dataset distributed throughout according to their DPW annotation. The protocol next describes how to obtain lists of differentially-expressed genes and save them into a text file (step 4.7), open the data table in Excel and perform various filtering steps in order to obtain the top ranked cluster markers for each cell cluster (step 4.8), and assign a new variable listing the top fibroblast marker genes named "FB_type_marker" (step 4.9). In step 4.10, the dot plot function is used to visualize the genes in the list in the fibroblast-only dataset by calling the "FB_type_marker" variable in the features parameter, and the representative results of this step (**Figure 9**) are dot plots confirming high expression of fibroblast subtype markers only in their respective cluster categories (top) but fairly distributed throughout DPW categories (bottom). In step 4.11, the same features variable is called to visualize the fibroblast marker genes in the overall wound healing dataset, and the representative result (**Supplementary Figure 11**) is a dot plot that confirmed the high expression of fibroblast subtype markers mostly in the original fibroblast. Finally, the following steps in the protocol used stacked bar plots to first visualize the proportions of DPW across the three fibroblast subtypes (step 4.12) and then to visualize the proportions of fibroblast subtypes across different time points (step 4.13). The representative results of these steps are proportion plots showing the relative numbers of DPW cells in each fibroblast subtype category (**Supplementary Figure 12**) and the relative numbers of fibroblast subtypes in each DPW category (**Supplementary Figure 13**). These results point to a significant change in fibroblast subtype proportions across the time course of healing, with the first fibroblast subtype (cluster 0) heavily dominant in early stage wounds (D1 and D3), the second subtype (cluster 1) dominant during wound resolution (D14), and the third subtype (cluster 2) being highest during the proliferative phase of wound healing (D7).

Beginning in method #5, the protocol walks through the steps for analyzing a single-cell wound healing dataset using the module scoring function in Seurat. The protocol first describes the steps of using a tab-deliminted text file to upload gene sets into variables in R (steps 5.1-5.2), followed by the application of the module scoring function to three gene sets relating to the three major phases of wound healing (step 5.3). In step 5.4, the dot plot function is used to visualize the aggregate module scores across two different metadata

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

categories, and the representative results for this step (**Figure 10**) are dot plots showing the average expression of the major healing phase modules across cells in the days post-wounding category (DPW, right) and across the major cell types category (left). These results show that the application of bulk sequencing-based gene expression profiles to single-cell expression datasets in a pseudo-bulk manner is a powerful method for comparative bioinformatics approaches by utilizing previously published datasets in the field of wound healing.

Beginning in method #6, the protocol walks through the steps for analyzing a Seurat-derived single-cell wound healing dataset using the CellChat package and workflow according to a specific scientific question of comparing cells derived from early- compared to late-phase wounds. The protocol first subsets the overall Seurat dataset into two time points post-injury, one during the inflammatory phase (day 1 (D1)) and the other during wound resolution (day 14 (D14)) (step 6.1). Two CellChat objects are created, and the protocol goes through all of the typical functions of the CellChat protocol to calculate all putative interactions between the cell types identified in method #3 of the protocol (steps 6.2-6.3). In step 6.4, the signaling scatter plot function is performed to visualize the incoming and outgoing interaction strengths in all major cell types at each wound healing time point. The representative results of this step (**Supplementary Figure 14**) are scatter plots that show the strengths of incoming (y-axis) and outgoing (x-axis) interactions for the major cell types at D1 (left) and D14 (right) time points. These results showed that immune cells such as neutrophils and macrophages had highest cell-cell interaction strengths during the inflammatory phase but fibroblasts dominated the cell-cell interactions during wound resolution, which confirms decades of wound healing research. The following steps focus the analysis on one of the significantly enriched pathways, the collagen pathway (steps 6.5-6.6). In step 6.7, the circle diagram function is performed to visualize the collagen signaling pathway interactions between cell types at the two time points. The representative results of this step (**Supplementary Figure 15**) are circle plots that show the inferred collagen pathway signaling interactions between all cell types at D1 (left) and D14 (right). In step 6.8, the same interactions are visualized using the chord diagram function, with the representative results (**Supplementary Figure 16**) being chord diagrams showing the inferred collagen pathway signaling interactions between all cell types at each time point. As expected, these results showed that fibroblasts were the primary source cells for the collagen signaling pathway, although the information flow was more restricted to immune cells at D1 compared to D14. To focus on the fibroblast as a source cell in the cell-cell interactions, step 6.9 repeats the chord diagram function adding a source cell parameter, and the representative results (**Supplementary Figure 17**) are chord diagrams showing the inferred collagen pathway signaling interactions with fibroblasts as source cells at each time point. In Step 6.10, two functions are performed to visualize the contributions of each ligand-receptor pair in the collagen signaling pathway with fibroblasts as source cells, one using bubble plots (step 6.10.1) and the other using chord diagrams (step 6.10.2). The representative results show the inferred contributions of each ligand-receptor pair in the collagen pathway signaling with fibroblasts as source cells at the D1 (left) and D14 (right) time points using both bubble plots (**Supplementary Figure 18**) and chord diagrams (**Supplementary Figure 19**). These results showed that at D1 the collagen pathway coming from fibroblasts was restricted to neutrophils and macrophages with a dominance of Cd44 and Sdc4 receptors, but in D14 other cells acted as receivers via a variety of

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

receptors including integrins. To focus on the Col1a1-Cd44 ligand-receptor interaction, which showed strong strengths in fibroblast interactions, a parameter is set (step 6.11) and then used in step 6.12 in a chord diagram function to visualize this particular ligand-receptor interaction between all cell types, with the representative results (**Supplementary Figure 20**) being chord diagrams showing the inferred Col1a1-Cd44 ligand-receptor interactions between all cell types at the D1 (left) and D14 (right) time points. These results showed that while in D1 this interaction is restricted to fibroblasts as source cells, in D14, macrophages and smooth muscle cells also act as source cells. Next, the protocol describes how to perform differential CellChat analysis by first merging the D1 and D14 CellChat objects (step 6.13). In step 6.14, the compare interactions function is performed to visualize the total numbers and relative strengths of cell-cell interactions between the two wound healing time points, and the representative results (**Supplementary Figure 21**) are the resulting bar plots showing the total numbers (left) and strengths (right) of inferred interactions in cells comprising D1 and D14 wounds, with higher numbers of interactions in D14 as opposed to higher relative strengths of interactions in D1. In steps 6.15 and 6.16, two functions are used to visualize the differential cell-cell interaction strengths between each cell type as the wound transitions from day 1 to day 14 with their respective representative results, the first being a circle plot (step 6.15, **Supplementary Figure 22**) and second being a heatmap (step 6.16, **Supplementary Figure 23**), where increased interactions in D14 compared to D1 shown in red and those that are decreased shown in blue. As expected, neutrophil- and macrophage-mediated interactions are increased in D1, and fibroblast-mediated interactions are increased in D14. In step 6.17, the ranking function is used to create a plot that ranks the relative contributions of individual pathways to cell-cell interactions with fibroblasts as source cells at D14 compared to D1, and the representative results (**Supplementary Figure 24**) show the resulting rank plot with D1 represented on the top in red and D14 on the bottom in blue, with several pathways being exclusively represented in either D1 or D14 and many others showing a gradient of activation. Finally, in step 6.18, two bubble plot functions are used to show the relative contributions of individual ligand-receptor pairs in the collagen signaling pathway with fibroblasts as source cells at D14 compared to D1, with the corresponding representative results (**Supplementary Figure 25**) showing increased (left) and decreased (right) signaling pairs in D14 compared to D1 across the many cell-cell interactions on the x-axis. As expected, fibroblasts had many more increased outgoing ligand-receptor pair interactions across several receiver cells in D14 wounds compared to D1 wounds, where the communication was more limited toward neutrophils and macrophages during the inflammatory phase.

Beginning in method #7, the protocol walks through the steps for integrating two single-cell wound healing datasets using Seurat. The protocol first describes the steps for merging two batches of the published single-cell datasets and applying the standard Seurat workflow to the merged dataset (steps 7.1-7.4). In step 7.5, the dimensional plot function is used to visualize the UMAP plot according to cluster and batch numbers of the merged but not-yet-integrated wound healing dataset. The representative results from this step (**Supplementary Figure 26**) are UMAP plots that visualize the distribution of Seurat clusters (left) and batch numbers (right), showing that there does not appear to be any significant batch effects for these two datasets before data integration. The protocol then performs data integration using the RPCA method and the follow-up Seurat workflow of the integrated dataset (steps 7.7-7.8). In step

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

7.9, the dimensional plot function is used to visualize the UMAP plot according to cluster and batch numbers of the integrated wound healing dataset. The representative results from this step (**Supplementary Figure 27**) are UMAP plots that visualize the distribution of Seurat clusters (left) and batch numbers (right), showing that there was now even greater overlap between the two batches across different clusters. The results also show the emergence of an additional cluster following integration of the data, which may point to the increased ability to identify potentially significant cell subtypes after the technical effects of data batches are controlled for.

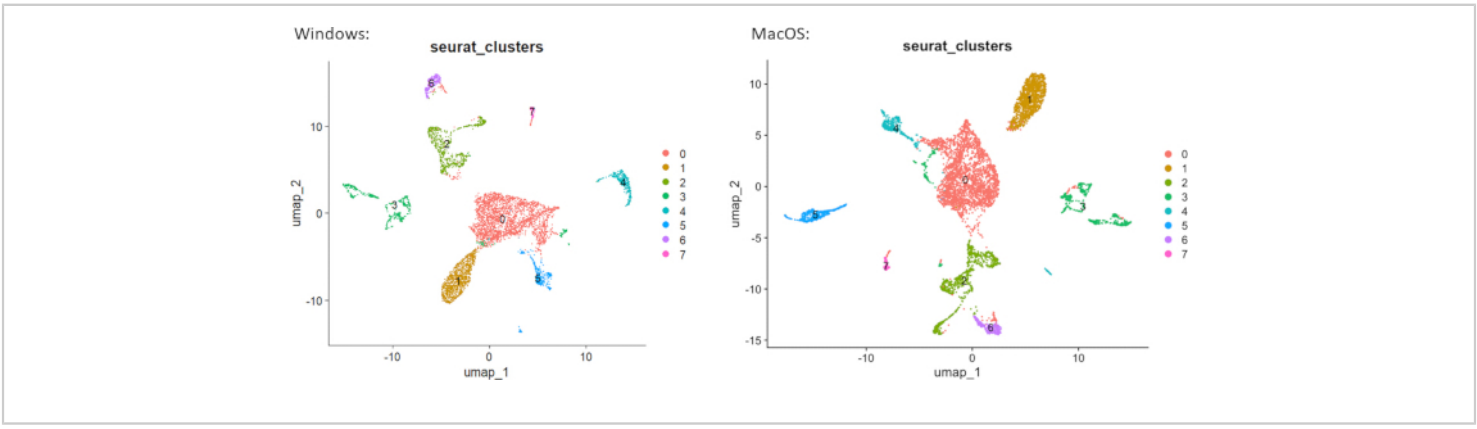

Figure 1: UMAP plot showing all cells in the dataset clustered around 8 major color-coded cluster groups. Results obtained from a computer running Windows (left) and MacOS (right). This figure corresponds to step 3.5. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

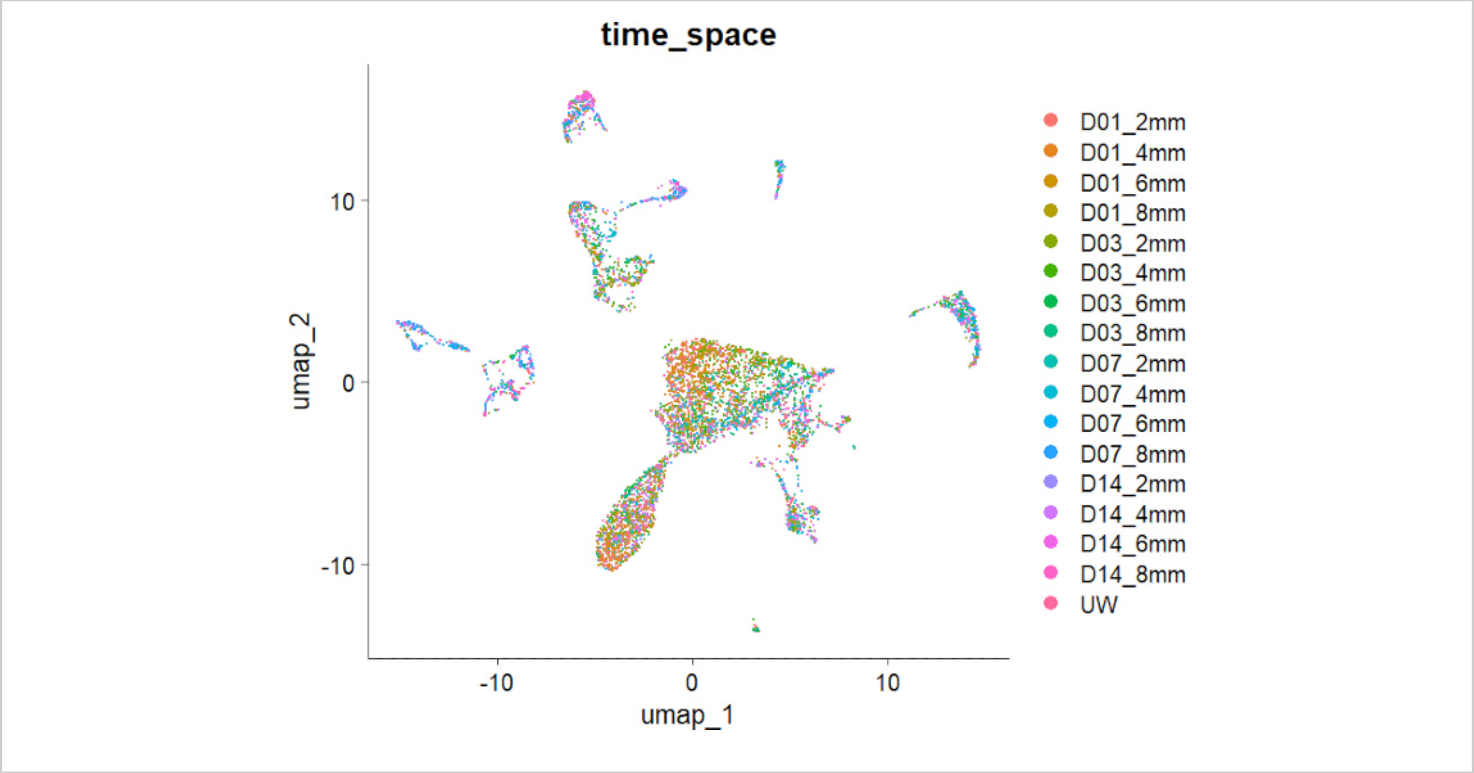

Figure 2: UMAP plot showing all cells in the dataset spread out according to their time/space origin, with no apparent clustering according to the time/space annotation. This figure corresponds to step 3.6. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

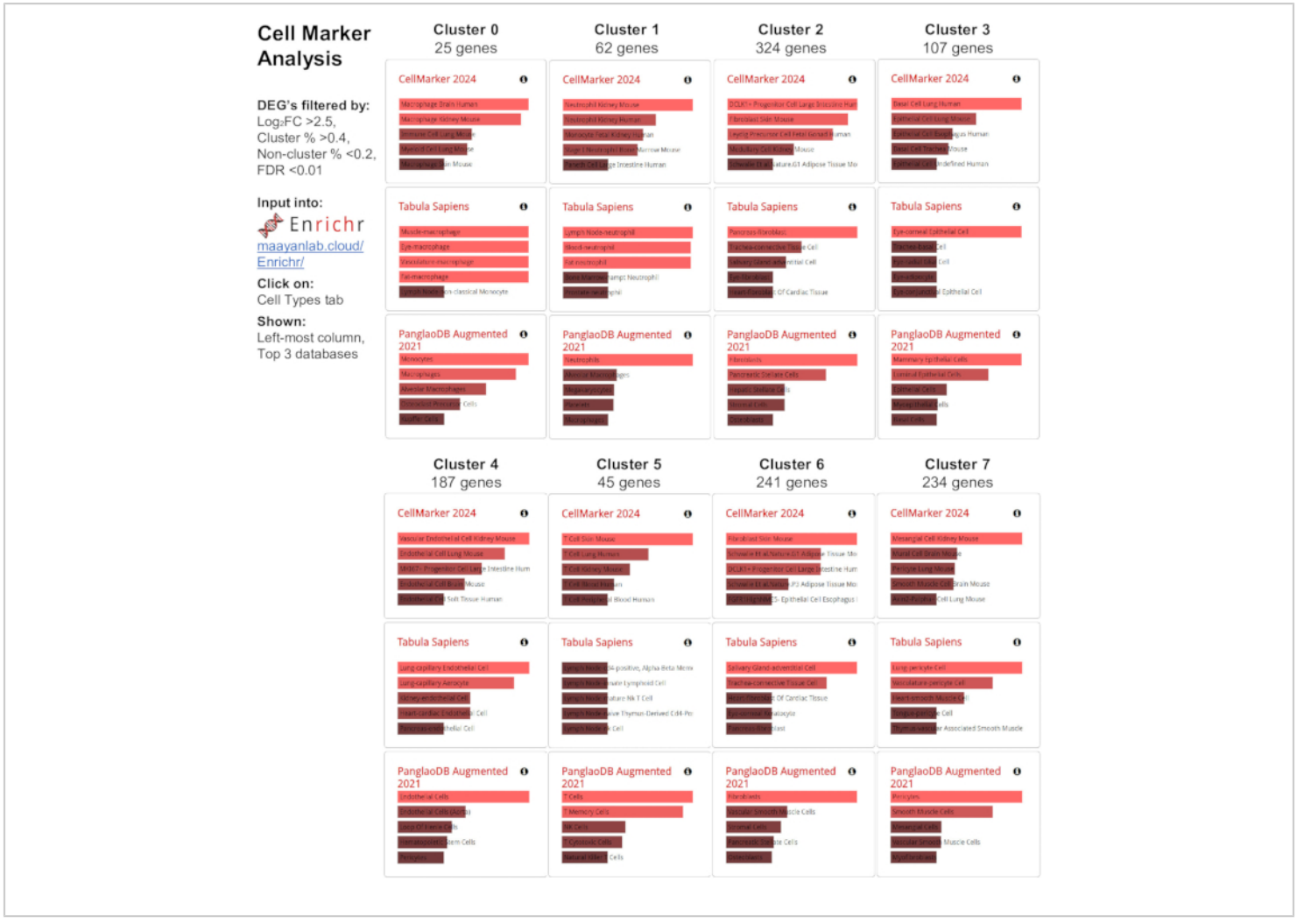

Figure 3: Cropped screenshots of the EnrichR outputs, showing the top enriched cell types for each cell cluster.

This figure corresponds to step 3.13. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

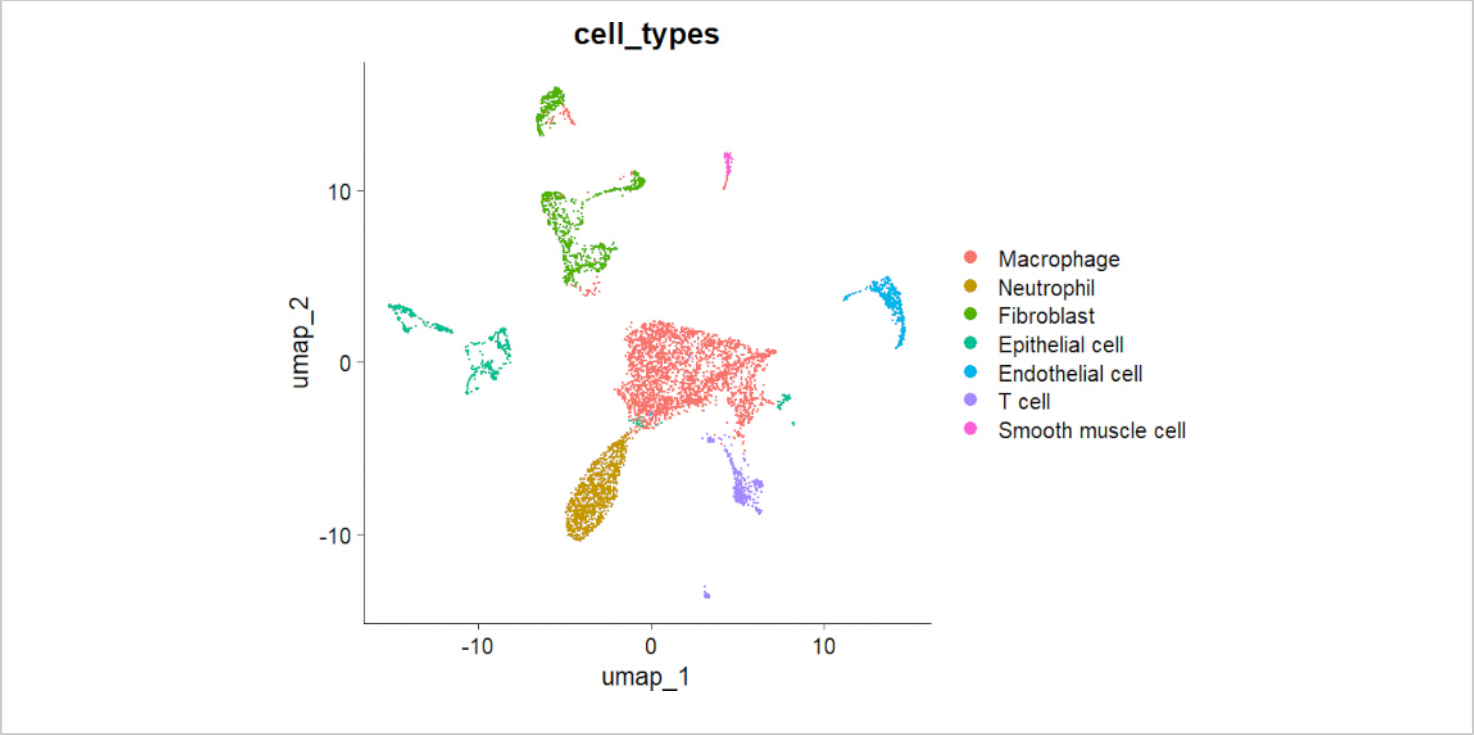

Figure 4: UMAP plot showing all cells in the dataset clustered around the major color-coded cell types. This figure corresponds to step 3.15. Please click here to view a larger version of this figure.

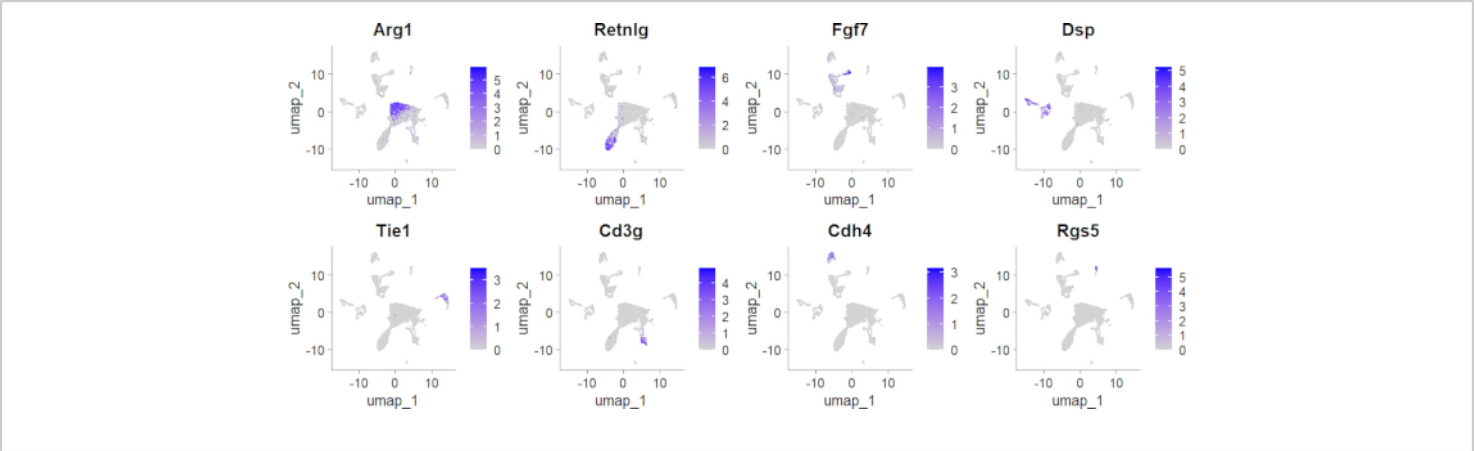

Figure 5: Grid of UMAP plots showing the high expression of the top cell marker genes within the major cell type clusters. This figure corresponds to step 3.16. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

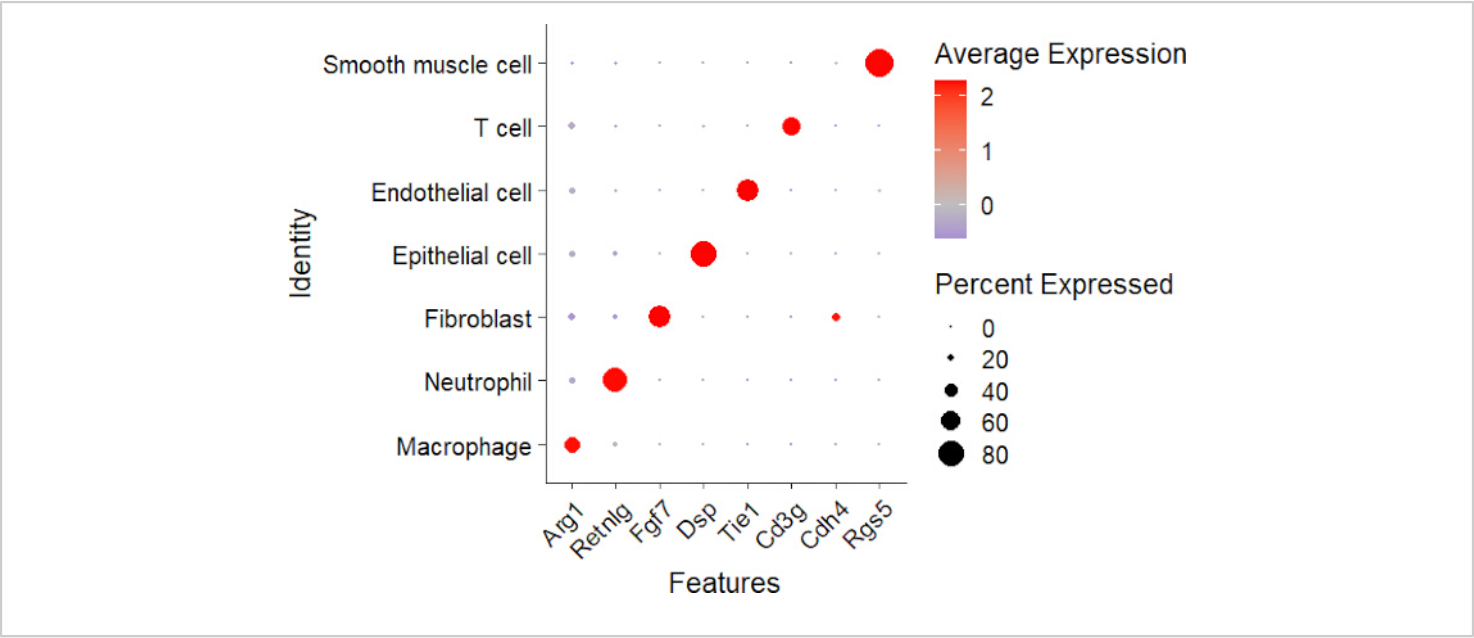

Figure 6: Dot plots confirming the high level of expression of the top cell marker genes only in their respective major cell types. This figure corresponds to step 3.18. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

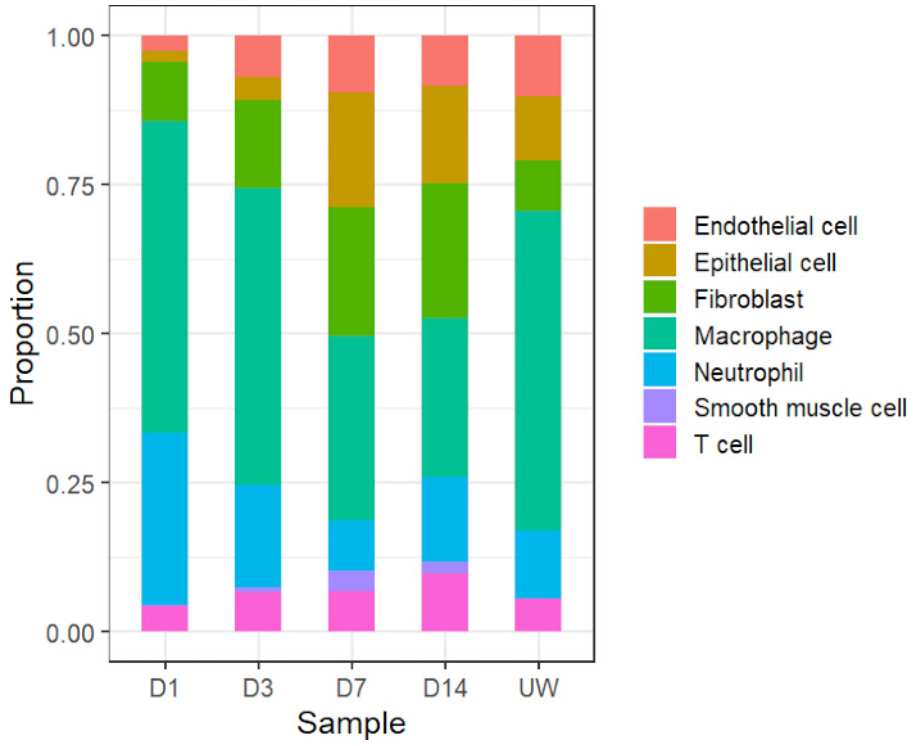

Figure 7: Proportion plot showing the relative numbers of major cell types in each DPW category. This figure corresponds to step 3.23. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

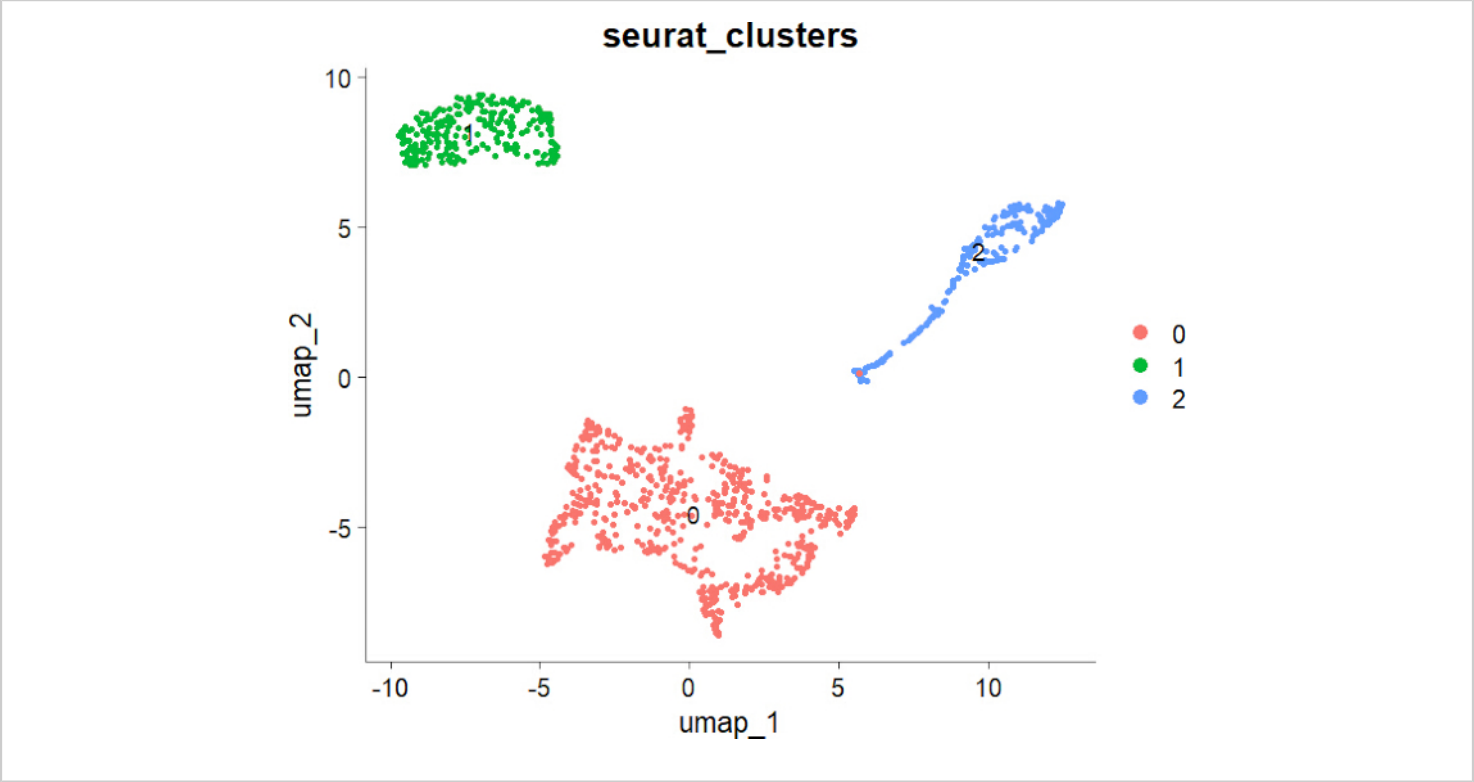

Figure 8: UMAP plot showing fibroblasts in the dataset clustered around the 3 color coded cell subtypes. This figure corresponds to step 4.5. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

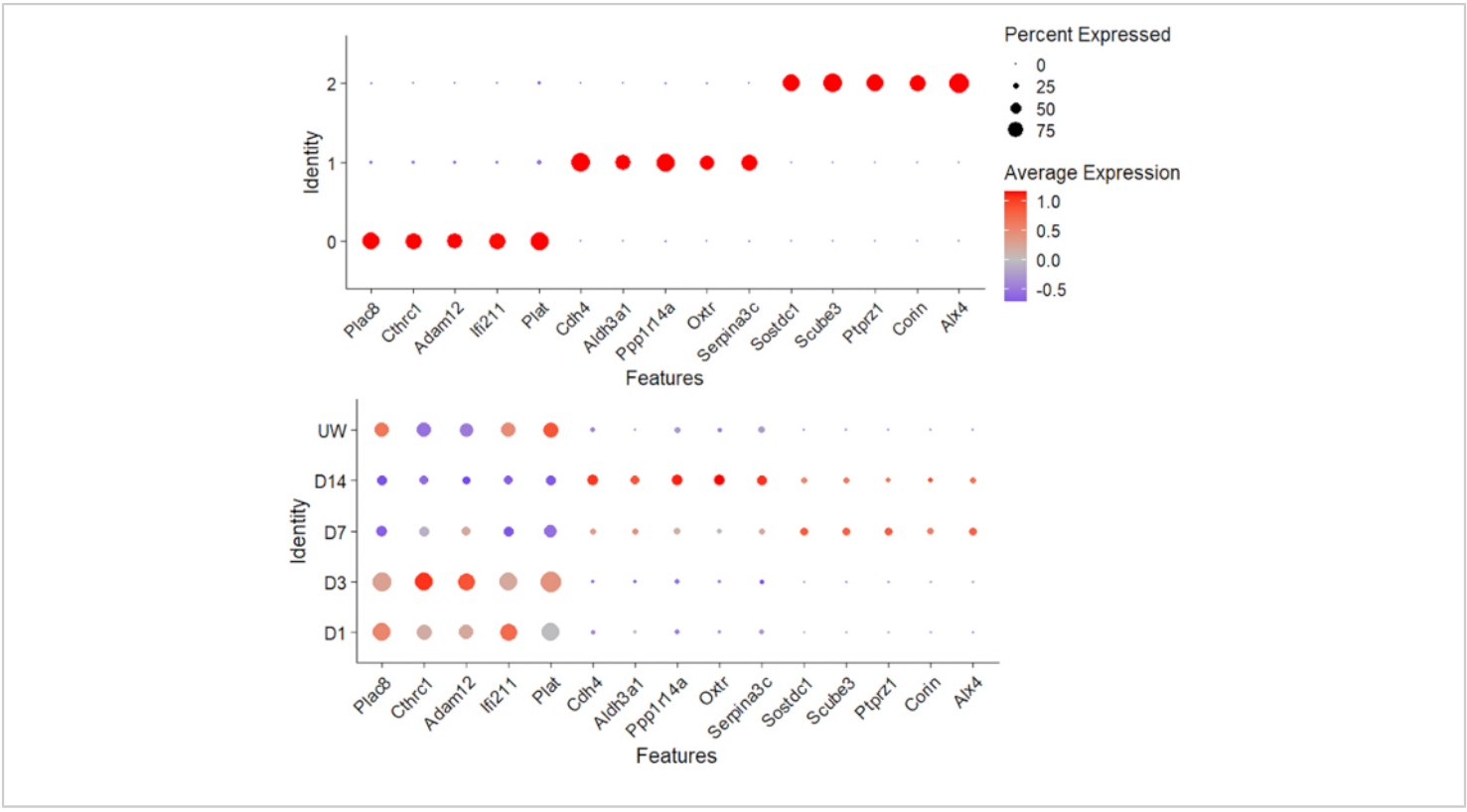

Figure 9: Dot plots confirming high expression of fibroblast subtype markers only in their respective cluster categories but fairly distributed throughout DPW categories. This figure corresponds to step 4.10. Please click here to view a larger version of this figure.

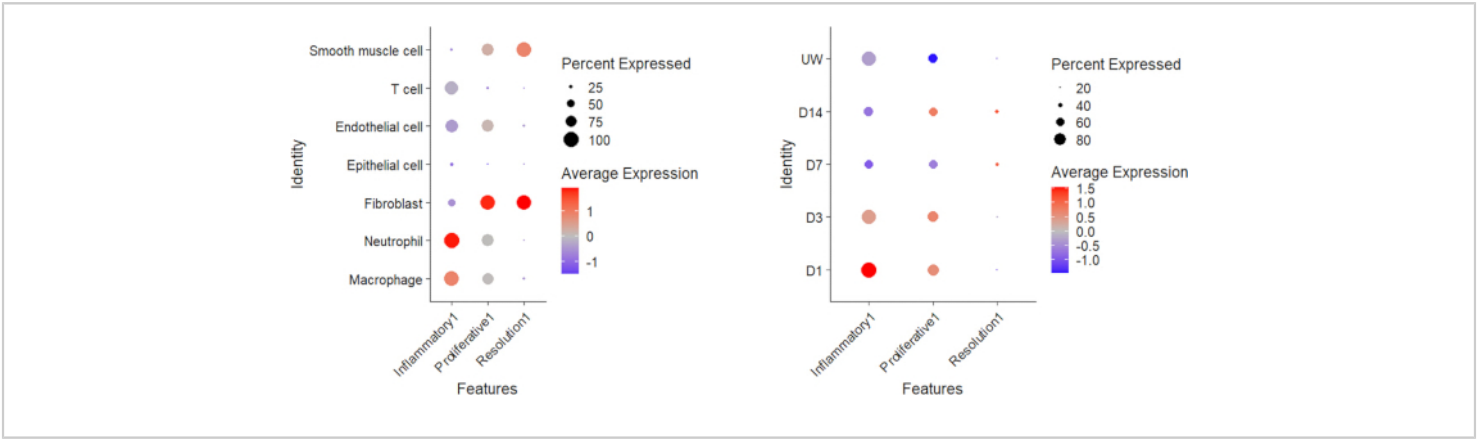

Figure 10: Dot plots showing the average expression of the major healing phase modules across cells per DPW and per major cell types. This figure corresponds to step 5.4. Please click here to view a larger version of this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

Supplementary Figure 1: Results showing that there is a fairly even distribution of detected genes for each barcode, which is important for dataset integrity and downstream analysis of wound healing timepoints. This figure corresponds to step 2.6.11. Please click here to download this figure.

Supplementary Figure 2: Scatter plots showing that there is a number of cells with large mitochondrial content, which correlates with low RNA counts --- these are dead or dying cells. This figure corresponds to step 2.10. Please click here to download this figure.

Supplementary Figure 3: Scatter plots showing that the distribution of detected genes and percentage mitochondrial RNA per cell is now more normal, clearing the way for robust downstream analyses. This figure corresponds to step 2.12.Please click here to download this figure.

Supplementary Figure 4: Violin plot showing that there are a number of cells with relatively high doublet score, and that 0.25 looks to be a natural cutoff, above which there is a population of likely doublets. This figure corresponds to step 2.15. Please click here to download this figure.

Supplementary Figure 5: Elbow plot showing that much of the major variation occurs within the first 13 dimensions. This figure corresponds to step 3.2. Please click here to download this figure.

Supplementary Figure 6: Dot plot confirming the high level of expression of the top cell marker genes only in their respective Seurat clusters. This figure corresponds to step 3.17. Please click here to download this figure.

Supplementary Figure 7: UMAP plot showing the localization of wound time-course annotations across the wound healing dataset. This figure corresponds to step 3.20. Please click here to download this figure.

Supplementary Figure 8: Proportion plot showing the relative numbers of DPW cells in each major cell type category. This figure corresponds to step 3.22.Please click here to download this figure.

Supplementary Figure 9: Elbow plot showing that much of the major variation in the fibroblast dataset occurs within the first 9 dimensions. This figure corresponds to step 4.2. Please click here to download this figure.

Supplementary Figure 10: UMAP plot showing fibroblasts in the dataset distributed throughout according to their DPW annotation. This figure corresponds to step 4.6. Please click here to download this figure.

Supplementary Figure 11: Dot plot confirming high expression of fibroblast subtype markers mostly in the original fibroblast cluster. This figure corresponds to step 4.11. Please click here to download this figure.

Supplementary Figure 12: Proportion plot showing the relative numbers of fibroblast subtypes in each DPW category. This figure corresponds to step 4.12. Please click here to download this figure.

Supplementary Figure 13: Proportion plot showing the relative numbers of fibroblasts across DPW in each fibroblast subtype category. This figure corresponds to step 4.13. Please click here to download this figure.

Supplementary Figure 14: Scatter plots showing the strengths of incoming (y-axis) and outgoing (x-axis)

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

interactions for the major cell types at day 1 (D1, left) and day 14 (D14, right) time points. This figure corresponds to step 6.4. Please click here to download this figure.

Supplementary Figure 15: Circle plots showing the inferred collagen pathway signaling interactions between all cell types in each DPW category. This figure corresponds to step 6.7. Please click here to download this figure.

Supplementary Figure 16: Chord diagrams showing the inferred collagen pathway signaling interactions between all cell types in each DPW category. This figure corresponds to step 6.8. Please click here to download this figure.

Supplementary Figure 17: Chord diagrams showing the inferred collagen pathway signaling interactions with fibroblasts as source cells in each DPW category. This figure corresponds to step 6.9. Please click here to download this figure.

Supplementary Figure 18: Bubble plots showing the inferred contributions of each ligand-receptor pair in the collagen pathway signaling with fibroblasts as source cells in each DPW category. This figure corresponds to step 6.10.1. Please click here to download this figure.

Supplementary Figure 19: Chord diagrams showing the inferred contributions of each ligand-receptor pair in the collagen pathway signaling with fibroblasts as source cells in each DPW category. This figure corresponds to step 6.10.2. Please click here to download this figure.

Supplementary Figure 20: Chord diagrams showing the inferred Col1a1-Cd44 ligand-receptor interactions between all cell types in each DPW category. This figure corresponds to step 6.12. Please click here to download this figure.

Supplementary Figure 21: Bar plots showing the number (left) and strength (right) of inferred interactions in day 1 and day 14 wounds. This figure corresponds to step 6.14. Please click here to download this figure.

Supplementary Figure 22: Circle plot showing the differential cell-cell interaction strengths between each cell type as the wound transitions from day 1 (blue) to day 14 (red) DPW. This figure corresponds to step 6.15. Please click here to download this figure.

Supplementary Figure 23: Heatmap showing the differential cell-cell interaction strengths between each cell type as the wound transitions from day 1 (blue) to day 14 (red) DPW. This figure corresponds to step 6.16. Please click here to download this figure.

Supplementary Figure 24: Rank plot showing the relative contributions of individual pathways to cell-cell interactions between fibroblasts and other cell types at day 1 vs day 14 DPW. This figure corresponds to step 6.17. Please click here to download this figure.

Supplementary Figure 25: Bubble plots showing the relative contributions of individual ligand-receptor pairs in the collagen signaling pathway with fibroblasts as source cells at day 1 vs day 14 DPW. This figure corresponds to step 6.18. Please click here to download this figure.

Supplementary Figure 26: UMAP plots showing distribution of Seurat clusters (left) and batch numbers (right) before data integration. This figure corresponds to step 7.5. Please click here to download this figure.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

Supplementary Figure 27: UMAP plots showing distribution of Seurat clusters (left) and batch numbers (right) after data integration. This figure corresponds to step 7.9. Please click here to download this figure.

Supplementary File 1: JoVE_Rscript.R: Main R code script file, which includes all of the steps and explanations described for all parts of the protocol. Please click here to download this file.

Supplementary File 2: JoVE_PhaseSpecificGenes.txt. Tab-delimited text file, which contains the lists of genes that are loaded in step 5.1 of the protocol. Please click here to download this file.

Supplementary File 3: JoVE_Rscript_b3.R. Supplementary R code script file, which includes all of the steps and explanations required to analyze batch #3 of the dataset for use in step 7.1 of the protocol. Please click here to download this file.

Supplementary Table 1: JoVE_DEGs_cellMarkers.xlsx. Excel file, which contains the full output of ranked differentially-expressed genes used in step 3.10 of the protocol. Please click here to download this table.

Supplementary Table 2: Top 5 up-regulated and expressed genes for each seurat cluster. Please click here to download this table.

Discussion

In this protocol, RStudio is used to run prescribed lines of code that enable a basic analysis of a complex single-cell dataset using Seurat. Several methods are presented that are relevant for wound healing research, including installation of the R coding environment, downloading a previously published single-cell wound healing dataset, performing critical quality control steps and standard single-cell analysis workflows including visualizations, major cell type annotations, cell subtype analyses, and integrative analyses using Seurat, and performing cell-cell interaction analyses using CellChat.

The methods introduced herein are simplified vignettes of typical workflows for single-cell analysis using R and its popular open-source scientific packages, Seurat[21] and CellChat[22]. Indeed, the workflow is only one example of the type of analysis one may accomplish with a complex single-cell wound healing dataset. The possible modifications to this method are nearly infinite, with the only restraint being the user's particular scientific inquiry. For example, the user may change some of the key parameters, such as cell types and time-points, according to the research questions that they may wish to ask of this dataset. The authors also hope that the user feels comfortable enough to adapt this workflow to their own single-cell dataset of interest; however, care must be taken when using this workflow to analyze other datasets, since each experiment may propagate technical and sample preparation issues to the data itself. Therefore, it is imperative that the user reads and understands all of the experimental details before interpreting the results of any previously published and re-analyzed single-cell datasets. It is important to remember that bioinformatics tools are a powerful method for the exploration of biological process and hypothesis generation, and that any critical biological interpretations of results must be validated in follow-up experiments.

Throughout the protocol, note that major modifications may be made to particular areas of the workflow to accomplish other tasks. However, the details of all possible combinations of modifications to the workflow are beyond the scope of

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

this manuscript. For example, the resolution used for cell clustering and the dimensions used for UMAP analysis are necessarily subjective, and the tools presented here allow for both large-scale analyses (as was demonstrated here for broadly-defined major cell types) and very specific analyses that could entail the sub-clustering of cells into more rare subpopulations within the larger dataset. For more information on this aspect of the single-cell analysis method, and for details about all the other parameters that may be changed in the single-cell analysis pipeline, the authors refer the user to the Seurat publications[21,26] and website (https://satijalab.org/seurat/), where the authors of this evolving tool provide in-depth explanations, vignettes, and tutorials.

This manuscript introduced some of the most cited and used tools in the single-cell transcriptomics literature, namely Seurat[21] and CellChat[22], for single-cell and cell-cell interaction analyses, respectively. However, other tools exist that perform similar functions in slightly different ways. For single-cell dataset analysis, there are Scran[27], Scater [28], and the Python-based ScanPy[29], which use various methods for dataset integration[25]. In this protocol, manual annotation of cell types was demonstrated, which relies on the user's judgement for interpreting cluster cell marker enrichments, but there now exist various tools enabling automated classification of cell types, such as SingleR[30] and scGate[31], among others. For cell-cell communication analyses, CellChat was demonstrated in this protocol, but there exist other tools for estimating cell-cell communications, including CellPhoneDB[32], Cytotalk[33], and other ligand-receptor databases that are implemented within the LIANA (LIgand-receptor ANalysis framework) consensus framework[34]. All bioinformatics tools are unique and come with their own peculiarities and modifiable parameters. Therefore, it is important for the user to carefully read each tool's associated documentation in order to understand its nuances before interpreting any output generated from their use. Finally, whatever bioinformatics tools one uses, it is critical to remember that such tools are continually evolving and that different versions of the packages may yield different outputs.

In R, syntax is critical, and a misplaced punctuation, quotation mark, bracket, or even a wrongly capitalized letter will result in an error. Therefore, it is vital for the user to pay attention to details when typing code and to be especially mindful when copying lines of code in order to adapt it to new scientific questions and datasets. For troubleshooting the specific errors one may encounter, the authors recommend simply copy-pasting the error message into the user's favorite web search engine and browsing through the results from bioinformatics forums such as GitHub and Stack Overflow, as the most commonly encountered errors have already likely been answered by a knowledgeable power user. In some forums, the most successful answers are 'upvoted' by other users who have found the solution to be best for the problem. The user must be careful not to simply copy-paste lines of code they found on the internet into their own computer (especially if a solution calls for changing one's system settings outside of the R programming language), since there is a chance that such programs may be malicious. An exciting emerging method of troubleshooting coding errors is using the powerful generative large language AI models such as OpenAI's ChatGPT, Microsoft's Copilot, or Google's Gemini. These models have proven to be especially useful for software engineering in general and troubleshooting in particular. For this, the user may copy-paste entire lines of their code after providing a simple prompt to the chatbot about the user's intention for the code. The usual caveat exists that these models are not foolproof, and the user may have to

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

try multiple prompts in order to generate an answer that is appropriate for solving the problem.

Disclosures

The authors have no conflicts of interest to disclose.

Acknowledgments

The laboratory of M.S. Wietecha received funding from the NIH/NIGMS grant R35-GM154921, the Wound Healing Society Research Grant, and the Department of Oral Biology at the UIC College of Dentistry.

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

References

1. Eming, S. A., Martin, P., Tomic-Canic, M. Wound repair and regeneration: mechanisms, signaling, and translation. *Sci Transl Med.* **6** (265), 265sr6 (2014).
2. Wietecha, M. S. et al. Phase-specific signatures of wound fibroblasts and matrix patterns define cancer-associated fibroblast subtypes. *Matrix Biol.* **119**, 19-56 (2023).
3. Rodrigues, M., Kosaric, N., Bonham, C. A., Gurtner, G. C. Wound healing: a cellular perspective. *Physiol Rev.* **99** (1), 665-706 (2019).
4. Chen, L., Mirza, R., Kwon, Y., DiPietro, L. A., Koh, T. J. The murine excisional wound model: contraction revisited. *Wound Repair Regen.* **23** (6), 874-877 (2015).
5. Rhea, L., Dunnwald, M. Murine excisional wound healing model and histological morphometric wound analysis. *J Vis Exp.* **162**, e61616 (2020).
6. Fischer, K. S. et al. Protocol for the splinted, human-like excisional wound model in mice. *Bio-Protocol.* **13** (3), e4606 (2023).
7. Iglesias-Bartolome, R. et al. Transcriptional signature primes human oral mucosa for rapid wound healing. *Sci Transl Med.* **10** (451), aap8798 (2018).
8. Chen, L., Arbieva, Z. H., Guo, S., Marucha, P. T., Mustoe, T. A., DiPietro, L. A. Positional differences in the wound transcriptome of skin and oral mucosa. *BMC Genomics.* **11**, 471 (2010).
9. Leonardo, T. R. et al. Transcriptional changes in human palate and skin healing. *Wound Repair Regen.* **31** (2), 156-170 (2023).
10. Rognoni, E. et al. Fibroblast state switching orchestrates dermal maturation and wound healing. *Mol Syst Biol.* **14** (8), e8174 (2018).
11. Bergmeier, V. et al. Identification of a myofibroblast-specific expression signature in skin wounds. *Matrix Biol.* **65**, 59-74 (2018).
12. Plikus, M. V. et al. Regeneration of fat cells from myofibroblasts during wound healing. *Science.* **355** (6326), 748-752 (2017).
13. Shook, B. A. et al. Myofibroblast proliferation and heterogeneity are supported by macrophages during skin repair. *Science.* **362** (6417), aar2971 (2018).
14. Rinkevich, Y. et al. Skin fibrosis. Identification and isolation of a dermal lineage with intrinsic fibrogenic potential. *Science.* **348** (6232), aaa2151 (2015).
15. Guerrero-Juarez, C. F. et al. Single-cell analysis reveals fibroblast heterogeneity and myeloid-derived adipocyte progenitors in murine skin wounds. *Nat Commun.* **10** (1), 650 (2019).
16. Gay, D. et al. Phagocytosis of Wnt inhibitor SFRP4 by late wound macrophages drives chronic Wnt activity for fibrotic skin healing. *Sci Adv.* **6** (12), eaay3704 (2020).
17. Haensel, D. et al. Defining epidermal basal cell states during skin homeostasis and wound healing using single-cell transcriptomics. *Cell Rep.* **30** (11), 3932-3947.e6 (2020).
18. Phan, Q. M., Sinha, S., Biernaskie, J., Driskell, R. R. Single-cell transcriptomic analysis of small and large wounds reveals the distinct spatial organization of regenerative fibroblasts. *Exp Dermatol.* **30** (1), 92-101 (2021).

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License

19. Foster, D. S. et al. Integrated spatial multiomics reveals fibroblast fate during tissue repair. *Proc Natl Acad Sci U S A.* **118** (41), e2110025118 (2021).
20. Hu, K. H. et al. Transcriptional space-time mapping identifies concerted immune and stromal cell patterns and gene programs in wound healing and cancer. *Cell Stem Cell.* **30** (6), 885-903.e10 (2023).
21. Hao, Y. et al. Dictionary learning for integrative, multimodal and scalable single-cell analysis. *Nat Biotechnol.* **42** (2), 293-304 (2024).
22. Jin, S. et al. Inference and analysis of cell-cell communication using CellChat. *Nat Commun.* **12** (1), 1088 (2021).
23. Germain, P.-L., Lun, A., Garcia Meixide, C., Macnair, W., Robinson, M. D. Doublet identification in single-cell sequencing data using scDblFinder. *F1000Research.* **10**, 979 (2021).
24. Jin, S., Plikus, M. V., Nie, Q. CellChat for systematic analysis of cell-cell communication from single-cell transcriptomics. *Nat Protoc.* **20** (1), 180-219 (2024).
25. Luecken, M. D. et al. Benchmarking atlas-level data integration in single-cell genomics. *Nat Methods.* **19** (1), 41-50 (2022).
26. Hao, Y. et al. Integrated analysis of multimodal single-cell data. *Cell.* **184** (13), 3573-3587.e29 (2021).
27. Lun, A. T. L., McCarthy, D. J., Marioni, J. C. A step-by-step workflow for low-level analysis of single-cell RNA-seq data with Bioconductor. *F1000Research.* **5**, 2122 (2016).
28. McCarthy, D. J., Campbell, K. R., Lun, A. T. L., Wills, Q. F. Scater: pre-processing, quality control, normalization and visualization of single-cell RNA-seq data in R. *Bioinformatics (Oxford, England).* **33** (8), 1179-1186 (2017).
29. Wolf, F. A., Angerer, P., Theis, F. J. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biol.* **19** (1), 15 (2018).
30. Aran, D. et al. Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage. *Nat Immunol.* **20** (2), 163-172 (2019).
31. Andreatta, M., Berenstein, A. J., Carmona, S. J. scGate: marker-based purification of cell types from heterogeneous single-cell RNA-seq datasets. *Bioinformatics.* **38** (9), 2642-2644 (2022).
32. Efremova, M., Vento-Tormo, M., Teichmann, S. A., Vento-Tormo, R. CellPhoneDB: inferring cell-cell communication from combined expression of multi-subunit ligand-receptor complexes. *Nat Protoc.* **15** (4), 1484-1506 (2020).
33. Hu, Y., Peng, T., Gao, L., Tan, K. CytoTalk: de novo construction of signal transduction networks using single-cell transcriptomic data. *Sci Adv.* **7** (16), eabf1356 (2021).
34. Dimitrov, D. et al. Comparison of methods and resources for cell-cell communication inference from single-cell RNA-Seq data. *Nat Commun.* **13** (1), 3224 (2022).

Copyright © 2025 JoVE Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License